

A project report on

**AUTOMATED DISEASE IDENTIFICATION IN
TEA AND BETEL LEAF CROPS USING
IMPROVISED MOBILENET V3**

Submitted in partial fulfilment of the requirements for the degree of

MASTER OF SCIENCE IN DATA SCIENCE

by

VIVEK SHARMA (23MSD7013)



**DEPT. OF MATHEMATICS, SCHOOL OF ADVANCED
SCIENCES, VIT-AP UNIVERSITY, ANDHRA PRADESH-
522241**

May, 2025

AUTOMATED DISEASE IDENTIFICATION IN TEA AND BETEL LEAF CROPS USING IMPROVISED MOBILENET V3

Submitted in partial fulfilment of the requirements for the degree of

MASTER OF SCIENCE IN DATA SCIENCE

by

VIVEK SHARMA (23MSD7013)



**DEPT. OF MATHEMATICS, SCHOOL OF ADVANCED
SCIENCES, VIT-AP UNIVERSITY, ANDHRA PRADESH-
522241**

May, 2025

DECLARATION

I hereby declare that the thesis entitled “**Automated Disease Identification in Tea and Betel Leaf Crops Using Improvised Mobilenet V3**” submitted by me, for the award of the degree of **Master of Science in Data Science** to VIT-AP is a record bonafide work carried out by me under the supervision of **Dr. Yada Nandukumar**.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Amaravati

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the Senior Design Project (SDP) entitled ‘**Automated Disease Identification in Tea and Betel Leaf Crops Using Improvised Mobilenet V3**’ submitted by **VIVEK SHARMA (23MSD7013)**, School of Advanced Sciences, VIT-AP, for the degree of Master of Science in Data Science is a record of bonafide work carried out by him under my supervision, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this university or any other university. The project report fulfils the requirements and regulations of VIT and in my opinion meets the necessary standards for submission.

Dr. Yada Nandukumar

Guide

The thesis is satisfactory/unsatisfactory

Internal Examiner

External Examiner

Approved by

PROGRAM CHAIR

DEAN

M. Sc. Data Science

School of Advanced Sciences

ABSTRACT

Early and accurate detection of plant leaf diseases is vital for supporting small-scale farmers, especially in regions like Northeast India and Southeast Asia where tea and betel are commonly cultivated together. These farmers often lack access to expert agronomists and advanced diagnostic tools, making timely disease identification a significant challenge. This study aims to address this gap by developing a lightweight, mobile-friendly deep learning solution for automated classification of tea and betel leaf diseases.

Through an extensive review of current literature, we observed that MobileNetV3 stands out as a highly efficient and lightweight pre-trained model suitable for deployment on mobile and embedded devices. Additionally, modern attention mechanisms such as Coordinate Attention (CA) and Efficient Local Attention (ELA) have been shown to outperform the traditional Squeeze-and-Excitation (SE) blocks integrated in MobileNetV3. Leveraging these insights, we proposed a modified architecture—**MobileNetV3 + ELA**—which replaces the SE block with a more computationally efficient ELA module to improve feature aggregation while maintaining model compactness.

To validate our approach, we conducted comparative experiments using a curated dataset consisting of 10 disease classes (six tea and four betel leaf diseases), each containing approximately 1,000 high-quality images. The evaluation was carried out in two distinct phases for each model: **Phase 1** using the **baseline version**, and **Phase 2** with the **last 20 layers fine-tuned**. This two-phase setup was applied to MobileNetV3, MobileNetV3 + CA, MobileNetV3 + ELA, and MnasNet, while the Custom CNN was evaluated in a single phase due to its static architecture. Our proposed **MobileNetV3 + ELA** model achieved the **highest test accuracy of 98.51%** in its fine-tuned version, outperforming all other configurations in terms of both computational efficiency and classification performance.

This work underscores the potential of combining lightweight CNNs with tailored attention mechanisms for developing effective and deployable disease detection tools. The proposed system is especially beneficial for smallholder farmers by offering an accurate, fast, and cost-effective solution for real-time crop health monitoring.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to **Dr. Yada Nandukumar, Assistant Professor, Department of Mathematics, School of Advanced Sciences (SAS), VIT-AP**, for his constant guidance, continual encouragement, understanding. more than all, he taught me patience in my endeavour. My association with him is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Mathematics and Research.

I would like to express my gratitude to **Dr. G. Viswanathan, Dr. Sekar Viswanathan, Dr S. V. Kota Reddy, and Prof. S. Srinivas, School of Advanced Sciences (SAS)** for providing with an environment to work in and for his inspiration during the tenure of the course.

In jubilant mood I express ingeniously my whole-hearted thanks to **Dr. Faizan Danish, Program Coordinator**, all teaching staff and members working as limbs of our university for their not-self-centred enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project

Place: Amaravati

Name of the student

Date:

Vivek Sharma

TABLE OF CONTENTS

TABLE OF CONTENTS	vii
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
PROBLEM STATEMENT	xii
CHAPTER 1	
INTRODUCTION	
1.1 INTRODUCTION	1
1.2 OBJECTIVES	2
1.3 SCOPE OF THE PROJECT	3
1.4 APPLICATIONS	3
CHAPTER 2	
LITERATURE REVIEW	6
CHAPTER 3	
METHODOLOGY	
3.1 METHODOLOGY	8
3.2 DATASET PREPARATION	10
3.3 DATA PREPROCESSING	13
3.4 NEURAL NETWORK ARCHITECTURES AND ATTENTION MECHANISMS	19
CHAPTER 4	
EXPERIMENTAL RESULTS AND ANALYSIS	
4.1 TEST ENVIRONMENT AND PARAMETER SETTING	31
4.2 EVALUATION INDEX	32
4.3 COMPARATIVE EXPERIMENT OF TRANSFER LEARNING TRAINING METHODS	33
4.4 PERFORMANCE ANALYSIS OF MOBILENET V3 ELA MODEL	34
4.5 COMPARATIVE TEST OF DIFFERENT MODELS	39
4.6 MAKING PREDICTION FROM THE DATASET	40
CHAPTER 5	
CODE IMPLEMENTATION	
5.1 IMPORTING LIBRARIES	44

5.2 DATA PREPROCESSING: AUGMENTATION OF BETEL LEAVES	44
5.3 COMBINING TEA AND BETEL DATA INTO ONE	45
5.4 RESIZING AND MERGING WITH ACTUAL CLASS HANDLING	45
5.5 OBSERVING SOME OF THE SAMPLE IMAGES [ONE IMAGE FROM EACH CLASS]	46
5.6 DEFINING ELA	47
5.7 DEFINING INVERTED RESIDUAL BLOCK WITH ELA	48
5.8 BUILDING MOBILENETV3 WITH ELA-S MODULE [PROPOSED MODEL]	49
5.9 HELPER FUNCTION FOR FREEZING AND UNFREEZING LAYERS	50
5.10 TRAINING PHASE 1 [WITHOUT FREEZING LAYERS]	50
5.11 FUNCTION TO SHOW TRUE VS PREDICTED IMAGES AND GRAD-CAM VISUALIZATIONS	51
5.12 PHASE 1: TRAINING ONLY THE CLASSIFIER	51
5.13 EVALUATION AFTER PHASE 1	52
5.14 FINE TUNING PHASE 2 [FREEZING LAST 20 LAYERS]	53
5.15 PHASE 2 SUMMARY	53
5.16 OPTIMIZER WITH THE UPDATED TRAINABLE PARAMETERS	53
5.17 TRAINING PHASE 2	53
5.18 EVALUATION PHASE 2	54
5.19 PREDICTIONS	55
CHAPTER 6	
CONCLUSION AND FUTURE WORK	
6.1 CONCLUSION	56
6.2 LIMITATIONS AND FUTURE SCOPE	57
REFERENCES	59

LIST OF FIGURES

Figure 1: Proposed Workflow	9
Figure 2: Data Augmentation Techniques	14
Figure 3: Few examples of augmentation techniques	15
Figure 4: Dataset Overview	16
Figure 5: Dataset Distribution	17
Figure 6: Architecture of Squeeze and Excite Attention Mechanism	21
Figure 7: Architecture of Coordinate Attention (CA)	23
Figure 8: Architecture of Efficient Local Attention (ELA)	24
Figure 9: Architecture Diagram of MobileNetV3	25
Figure 10: Proposed Model (MobileNetV3 with ELA)	29
Figure 11: Accuracy curves of full migration method	34
Figure 12: Accuracy curves of fine-tuning method	34
Figure 13: Loss Value curves of the proposed model	35
Figure 14: Confusion Matrix of the proposed model	36
Figure 15: Few predicted as well as grad-cam visualization images by the model	41
Figure 16: Snapshots of web app-based prediction	42

LIST OF TABLES

Table 1: Sample images from dataset	11
Table 2: Mobilenet V3 Architecture	27
Table 3: Parameter Setting	31
Table 4: Performance of MobileNetV3-ELA trained by 2 different transfer methods	33
Table 5: Classification Report of the proposed model	38
Table 6: Comparison of accuracy, parameters and computation between models	39

LIST OF ABBREVIATIONS

▪ CNN	Convolutional Neural Network
▪ CA	Coordinate Attention
▪ ELA	Efficient Local Attention
▪ SE	Squeeze-and-Excitation
▪ IoT	Internet of Things
▪ GPU	Graphics Processing Unit
▪ TPU	Tensor Processing Unit
▪ ReLU	Rectified Linear Unit
▪ GAP	Global Average Pooling
▪ FC	Fully Connected (Layer)
▪ Adam	Adaptive Moment Estimation
▪ BN	Batch Normalization
▪ NBN	No Batch Normalization
▪ Bneck	Bottle Neck
▪ HS	Hard Swish
▪ RE	Rectified Linear Unit

PROBLEM STATEMENT

Tea and betel leaf cultivation are critical components of the agricultural economy in Northeast India and Southeast Asia. However, these crops are highly vulnerable to various diseases, which can significantly impact yield, farmer income, and crop quality. Early detection and treatment of diseases are crucial for sustainable agricultural practices.

Current Methods:

Farmers typically rely on manual visual inspection to detect diseases in tea and betel leaves. This method is often inaccurate, time-consuming, and dependent on the farmer's experience. It does not guarantee early-stage disease detection and is impractical for large-scale monitoring. Traditional image processing techniques have been explored but often fall short in terms of scalability, precision, and adaptability under varying field conditions.

Proposed Solution:

This project investigates the feasibility of using lightweight deep learning models, specifically MobileNetV3 enhanced with customized attention mechanisms, to automatically detect and classify diseases in tea and betel leaves. By developing an efficient AI-based system, the goal is to enable real-time, accurate disease detection even in low-resource rural environments.

Objective:

The project aims to:

- Propose a modified lightweight deep learning model—MobileNetV3 integrated with Efficient Local Attention (ELA)—for accurate classification of tea and betel leaf diseases.
- Compare the performance of the proposed MobileNetV3 + ELA model against other pre-trained and custom architectures, including MobileNetV3 (baseline and fine-tuned), MobileNetV3 + Coordinate Attention (CA) (baseline and fine-tuned), MnasNet, and a custom CNN.
- Identify a highly accurate and computationally efficient model suitable for deployment on mobile and edge devices.

- Empower smallholder farmers in regions like Northeast India and Southeast Asia with early, accessible, and cost-effective plant disease detection tools to enhance crop health, yield, and sustainability

Chapter 1

INTRODUCTION

1.1 INTRODUCTION

Tea and betel leaf farming constitute the primary agricultural livelihood for smallholder farmers in Northeast India and Southeast Asia. In regions like Assam, tea transcends its role as a beverage—it is an economic lifeline. India, the world’s second-largest tea producer, contributes 28% of global tea output. As per the Tea Board of India, the country yielded 1284.78 million kg of tea between January and December 2024, reflecting its monumental economic footprint. Assam alone supplies over 50% of India’s tea, with the sector underpinning employment and regional development. Similarly, betel leaf (commonly called Paan), consumed by 15–20 million people in India, is cultivated traditionally across 55,000 hectares, generating annual revenues of approximately ₹9000 million.

However, tea and betel leaf production face persistent threats from leaf diseases caused by fungi, bacteria, or viruses. These infections compromise yield and quality, often leading to severe financial setbacks for farmers if undetected early. Betel vines, frequently intercropped with tea in smallholder systems, are equally vulnerable to pathogens. Delayed identification of subtle disease symptoms exacerbates losses, as traditional treatment methods lose efficacy when applied late.

Conventional disease detection in these crops relies heavily on manual inspection by farmers—a labor-intensive, subjective, and error-prone process. Early-stage symptoms, such as faint discolorations or lesions, often evade human observation, especially in large plantations. This limitation underscores the need for scalable, automated solutions to enhance accuracy and timeliness in disease diagnosis.

To address these challenges, this project proposes a lightweight AI-based image classification system designed for simultaneous disease detection in tea and betel leaves. Tailored for resource-constrained farmers, the solution prioritizes computational efficiency without

compromising diagnostic precision. Recent advancements in lightweight convolutional architectures and attention mechanisms inspired the integration of MobileNetV3 with an Efficient Local Attention (ELA) module.

The ELA module replaces the default Squeeze-and-Excitation (SE) blocks in MobileNetV3, enabling superior cross-layer feature aggregation with minimal computational overhead. This innovation enhances model accuracy while retaining compatibility with mobile devices, making it ideal for real-time deployment in agricultural settings.

To validate the proposed MobileNetV3 + ELA framework, we conducted comparative experiments against baseline MobileNetV3, its fine-tuned variant (last 20 layers optimized), MobileNetV3 with Coordinate Attention (CA), MnasNet, and a custom 5-layer CNN. Each pre-trained model underwent two evaluation phases: Phase 1 (base architecture) and Phase 2 (fine-tuned layers), ensuring equitable assessment across architectures.

The fine-tuned MobileNetV3 + ELA model achieved state-of-the-art performance, attaining 98.51% test accuracy with only ~2 million parameters. These results underscore the effectiveness of attention-enhanced MobileNet architectures in plant disease detection. By offering a portable, cost-effective tool for early diagnosis, this system empowers smallholder farmers to mitigate crop losses and secure livelihoods, bridging the gap between agricultural practices and cutting-edge AI.

1.2 OBJECTIVES

- To propose and implement a modified lightweight deep learning architecture, MobileNetV3 + Efficient Local Attention (ELA), for accurate classification of tea and betel leaf diseases.
- To evaluate and compare the performance of the proposed model with MobileNetV3 (baseline and fine-tuned), MobileNetV3 + Coordinate Attention (CA), MnasNet, and a custom CNN architecture.

- To design an AI-powered solution that delivers high accuracy with low computational overhead, enabling deployment on mobile and edge devices.
- To support smallholder farmers with early and reliable disease detection tools that can help reduce crop losses and improve overall yield.
- To promote accessible, real-time, and sustainable agricultural disease management, especially in resource-limited rural and remote areas.

1.3 SCOPE OF THE PROJECT

- Focuses specifically on detecting multiple types of diseases affecting tea and betel leaves simultaneously.
- Targets small and medium-sized farming communities in Northeast India and Southeast Asia, where tea and betel cultivation are economically significant.
- Covers the training, validation, and testing of deep learning models on a curated dataset of diseased and healthy tea and betel leaf images.
- Involves the integration of customized attention mechanisms to boost the performance of lightweight models.
- The model is intended to be scalable for real-time deployment on smartphones, drones, or edge computing devices for field applications.

1.4 APPLICATIONS

1. Smart Agriculture:

The developed lightweight disease detection model can be directly implemented into mobile or handheld devices, enabling farmers to diagnose diseases in tea and betel leaves instantly. By simply capturing an image of the affected leaf, the system can predict the disease class accurately, helping farmers make immediate decisions on pest control and treatment. This promotes smarter agricultural practices and reduces dependency on manual inspection or delayed lab testing.

2. Agricultural Advisory Services:

Agricultural officers and field extension workers often struggle with accurately diagnosing plant diseases during field visits. This model can serve as a powerful tool for advisory services by providing quick and reliable disease identification support. It can enhance the quality of guidance given to farmers, thereby improving disease management strategies and reducing the chances of crop loss due to delayed intervention.

3. Crop Monitoring Systems:

The model can be integrated into drone systems or IoT-based crop monitoring setups. By analysing images collected from fields on a large scale, it can automatically detect early signs of disease outbreaks across vast tea and betel plantations. This not only helps in continuous health monitoring but also minimizes manual labor, speeds up surveillance processes, and allows for targeted treatments rather than blanket spraying.

4. Early Warning Systems:

An early disease detection system can act as a critical component of early warning platforms for tea and betel farmers. Once a disease is detected in its initial stages, the system can send alerts or notifications, prompting farmers to take preventive or remedial actions. Early intervention can significantly limit the spread of infections, safeguard the overall crop yield, and ensure food and economic security for farming communities.

5. Research and Development:

The automated system can aid agricultural researchers by providing a tool for large-scale disease data collection, classification, and analysis. Researchers can use this model to study the progression, spread patterns, and emergence of new diseases

affecting tea and betel crops. This can contribute to more informed research on disease-resistant plant varieties and effective pesticide usage strategies.

6. Agro-Insurance Support:

Crop insurance companies require accurate and timely evidence of disease or crop failure for processing claims. This disease detection model can serve as an authentic, automated, and reliable method for documenting plant health status. It can streamline claim validation processes, reduce the chance of fraudulent claims, and ensure that deserving farmers receive financial support promptly.

Chapter 2

LITERATURE REVIEW

Plant disease detection using deep learning and computer vision has gained substantial momentum over the past decade. Early studies, such as by Mohanty et al. [1], demonstrated the use of basic Convolutional Neural Networks (CNNs) to classify diseases across 14 different crop species, achieving remarkable accuracy. However, these models were computationally heavy and less suited for lightweight or mobile applications. Similarly, Sladojevic et al. [2] employed deep CNN models for fruit and vegetable disease detection but faced limitations regarding scalability and inference speed on resource-constrained devices.

Recognizing the need for more efficient models, researchers shifted towards lightweight architectures. Too et al. [3] systematically compared models like MobileNet, DenseNet, and InceptionV3 on the PlantVillage dataset, showing that MobileNet provided an optimal balance between accuracy and model size. This was further advanced by Howard et al. [4] with the introduction of MobileNetV3, which incorporated squeeze-and-excitation modules and hard-swish activation functions, resulting in even lighter and faster networks ideal for real-world deployment.

To further enhance the feature extraction capabilities without substantially increasing model size, attention mechanisms were introduced. Hou et al. [5] proposed the Coordinate Attention (CA) mechanism, which captures both channel relationships and positional information effectively. In parallel, Wang et al. [6] developed the Efficient Local Attention (ELA) module, enabling deeper networks to better aggregate features across layers with minimal computational burden.

Applications of deep learning models to specific plants and diseases have also expanded. Amara et al. [7] used CNNs for banana leaf disease detection, emphasizing the challenges

of varying environmental conditions and disease stages. Zhang et al. [8] further pushed the field by integrating EfficientNet with attention mechanisms to detect tomato diseases with high precision, showcasing how attention-augmented lightweight networks can boost performance on complex agricultural datasets.

While significant progress has been made, most of the existing work has focused on single crop types. Few attempts have addressed multi-crop disease detection simultaneously, especially for co-cultivated plants like tea and betel leaves, which are crucial to the economy of Northeast India and Southeast Asia. Moreover, limited studies have explored the customization of attention mechanisms specifically tailored to lightweight architectures for agricultural applications.

Therefore, the present study builds upon these advancements by developing a lightweight yet accurate model for simultaneous disease detection in tea and betel leaves. It leverages MobileNetV3, further enhanced by customized attention modules like Coordinate Attention (CA) and Efficient Local Attention (ELA), to maximize performance while maintaining model compactness suitable for practical deployment among farmers.

Chapter 3

METHODOLOGY

3.1 METHODOLOGY

The core objective of this study is to design and evaluate a lightweight yet highly accurate deep learning model for disease detection in tea and betel leaves. To achieve this, we propose an enhanced architecture based on MobileNetV3 integrated with Efficient Local Attention (ELA), referred to as MobileNetV3+ELA. This model is specifically tailored to improve multi-layer feature aggregation while maintaining computational efficiency—making it suitable for deployment in real-world agricultural settings, especially on mobile or low-resource devices.

We began by preparing the dataset, which includes images of diseased tea and betel leaves categorized into respective classes. Preprocessing steps included resizing all images to 256 * 256 pixels, applying data augmentation to balance the class distributions, normalizing pixel values, and splitting the dataset into training, validation, and test subsets. Label encoding was performed to convert class names into numerical labels suitable for model training.

The proposed MobileNetV3+ELA model was then implemented by customizing the MobileNetV3 backbone with an ELA module, which enhances its ability to capture important features across layers. This model was trained using the augmented dataset, with hyperparameters such as learning rate, batch size, and number of epochs optimized to maximize accuracy while preventing overfitting. Training and validation performance were closely monitored through accuracy and loss plots across epochs.

To assess the effectiveness of our proposed model, we conducted a comprehensive comparative analysis with several baseline and benchmark models. These included a custom Convolutional Neural Network (CNN) architecture built from scratch, along with popular

lightweight models such as MnasNet and MobileNetV3. For a thorough evaluation, we trained both the baseline and fine-tuned versions of MobileNetV3, as well as MobileNetV3 integrated with Coordinate Attention (MobileNetV3+CA). In total, the comparison involved MobileNetV3 (baseline and fine-tuned), MobileNetV3+CA (baseline and fine-tuned), our proposed MobileNetV3+ELA (baseline and fine-tuned), and the custom CNN model. All models were trained under identical experimental settings using the same dataset splits to ensure a fair and reliable performance comparison.

The final evaluation was performed on the test set using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix. The results of MobileNetV3+ELA were benchmarked against the other models to validate its superiority in terms of both classification performance and model compactness. This methodology ensures a robust comparison and establishes the practical relevance of the proposed model for real-time agricultural disease detection.

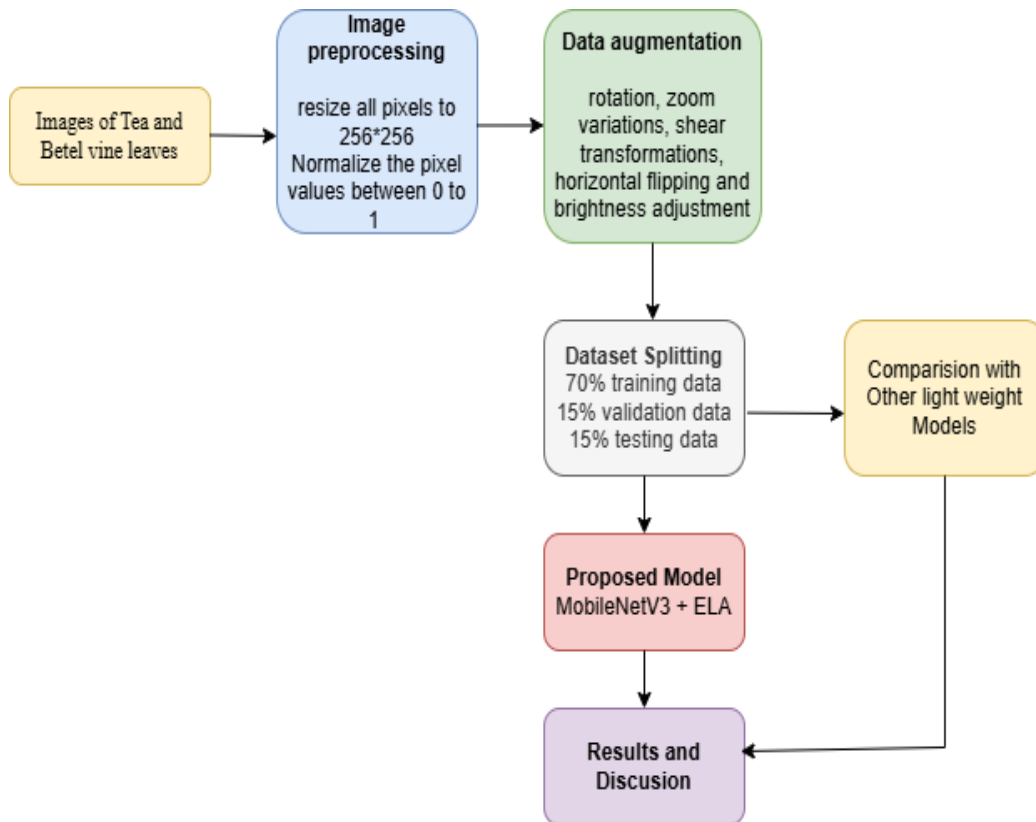


Figure 1: Proposed Workflow

3.2 DATASET PREPARATION

In this project, we have created a combined dataset by merging two distinct datasets: one for tea leaves and another for betel leaves. Both datasets were sourced from publicly available platforms and structured to support multi-class classification tasks related to plant disease detection.

Tea Leaf Dataset

The tea leaf dataset was downloaded from Kaggle, authored by Saikat Dutta. It consists of images categorized into six distinct classes, each representing a specific disease or healthy condition observed in tea plants.

The six classes in the tea leaf dataset are:

- tea_algal_spot
- tea_brown_blight
- tea_gray_blight
- tea_healthy
- tea_helopeltis
- tea_red_spot

Each class folder contains approximately **1000 images**, resulting in a well-balanced dataset. The images are of high quality and were specifically curated to facilitate the training of machine learning and deep learning models for accurate tea leaf disease detection.

Betel Leaf Dataset

The betel leaf dataset was sourced from Mendeley Data. It provides a categorized collection of betel leaf images based on four conditions:

- Healthy Betel Leaves

- Dried Leaves
- Bacterial Leaf Disease
- Fungal Brown Spot Disease

The dataset was captured in Camperhat, Raipur, Lakshmipur, Bangladesh with careful attention to diversity and quality.

We utilized the original folder, containing 1000 images equally distributed across the four classes (250 images per class).

This ensures a balanced and representative distribution to train and validate machine learning models effectively.

Combined Dataset Structure

For this project, both datasets were merged into a single dataset. The combined dataset consists of 10 classes in total (6 from tea leaves and 4 from betel leaves).

Table 1: Sample images from dataset

Algal Spot		
Brown Blight		
Grey Blight		

Healthy Tea Leaves		
Helopeltis		
Red Spot		
Bacterial Leaf Disease		
Fungal Brown Spot Disease		
Dried Leaf		

Healthy Betel
Leaves



3.3 DATA PREPROCESSING

Data preprocessing is one of the most critical steps in building a reliable deep learning model. It involves transforming raw image data into a clean, balanced, and standardized format that allows the model to learn effectively and generalize well. In this project, careful attention was given to preprocessing to address class imbalance, reduce bias, and ensure consistency across the dataset. Below are the steps performed, along with the reasoning behind each.

3.3.1 DATA AUGMENTATION

To ensure all classes had an equal number of images, data augmentation was applied exclusively to the betel leaf dataset. Initially, the dataset contained 250 images per class across four disease categories (totalling 1000 images), which was significantly lower than the tea leaf dataset, where each of the six disease classes already had approximately 1000 images. To address this, each original image in the betel leaf dataset was augmented exactly three times using a combination of transformations such as horizontal flipping, rotation, brightness adjustment, and slight zooming.

How augmentation bias was removed:

To prevent any form of augmentation bias and maintain consistency across dataset splits, each original image and its three augmented versions were placed in the same split folder (i.e., train, validation, or test). For instance, if the original image was assigned to the training set, its three augmented variants were also stored in the training folder. This ensured that no augmented version of a training image leaked into the validation or test sets, which could otherwise lead to artificially inflated evaluation scores due to data leakage.

Why it's important:

Class imbalance can lead to models biased toward majority classes, impairing their ability to generalize across all categories. By augmenting the betel dataset uniformly to reach 1000 images per class, we balanced the dataset and reduced the risk of biased learning. Moreover, by keeping augmented sets grouped with their original counterparts within the same split, we preserved the integrity of model evaluation, ensuring a robust and fair assessment of generalization capabilities.

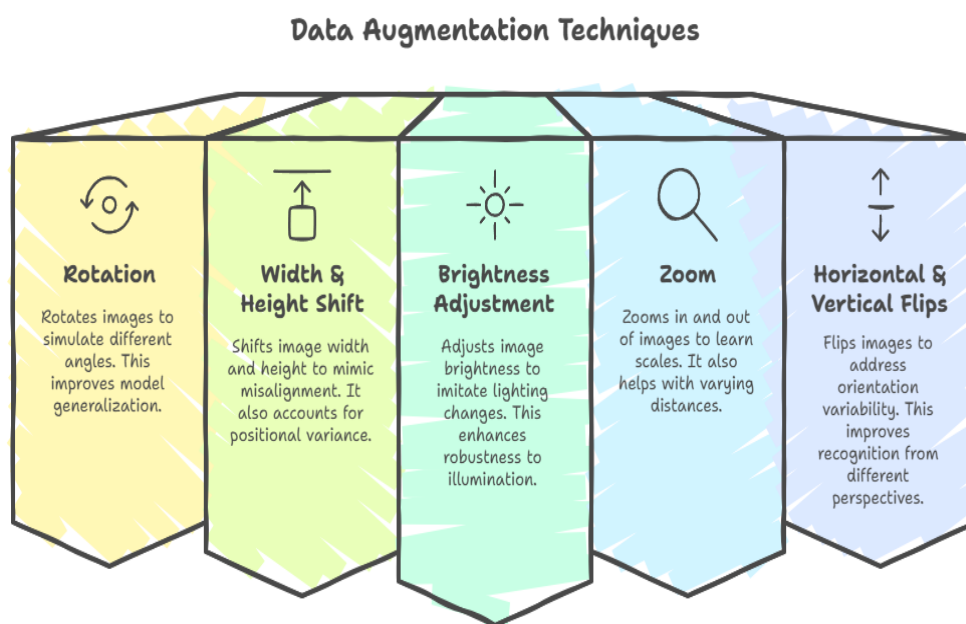


Figure 2: Data Augmentation Techniques

The augmentation techniques applied include:

- Rotation
- Horizontal and vertical flips
- Random zoom
- Brightness/contrast adjustment
- Random cropping and translation

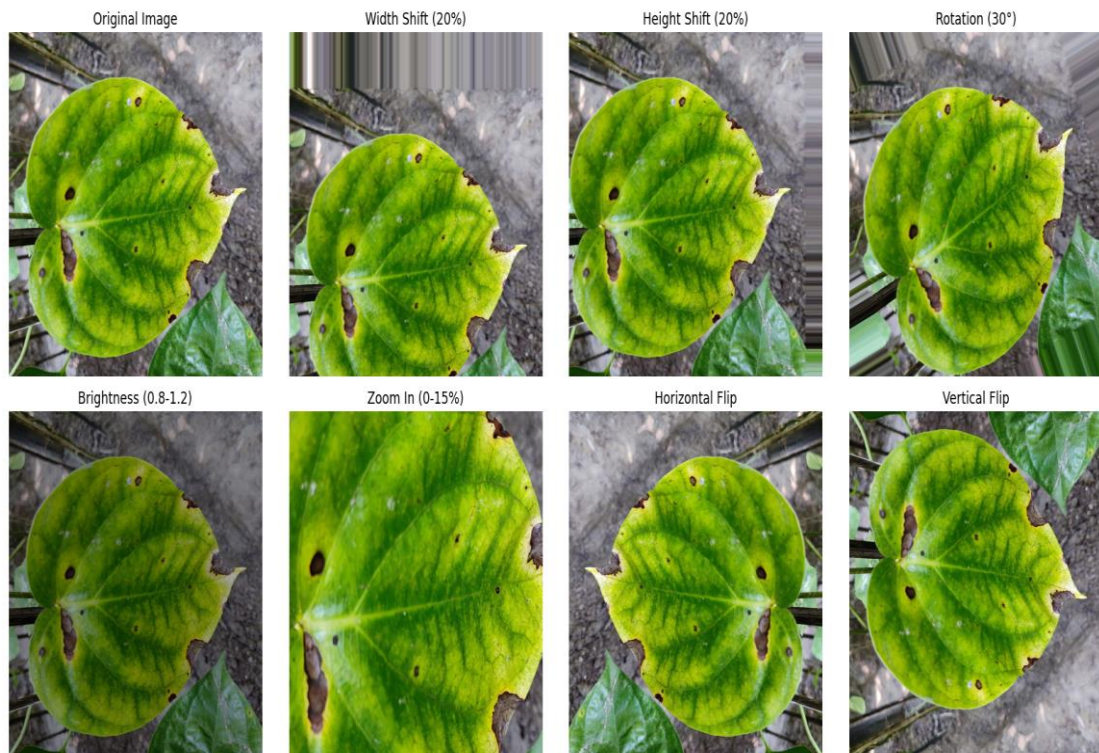


Figure 3: Few examples of augmentation techniques

As shown in the image, techniques like rotation, shifting, flipping, zooming, and brightness adjustment are applied to create diverse versions of the same image. This improves the model's ability to generalize and recognize patterns under different conditions, reducing overfitting and enhancing performance.

3.3.2 DATASET SPLITTING

The combined dataset (tea + betel) was divided into three subsets:

- Training set: 70%
- Validation set: 15%
- Testing set: 15%

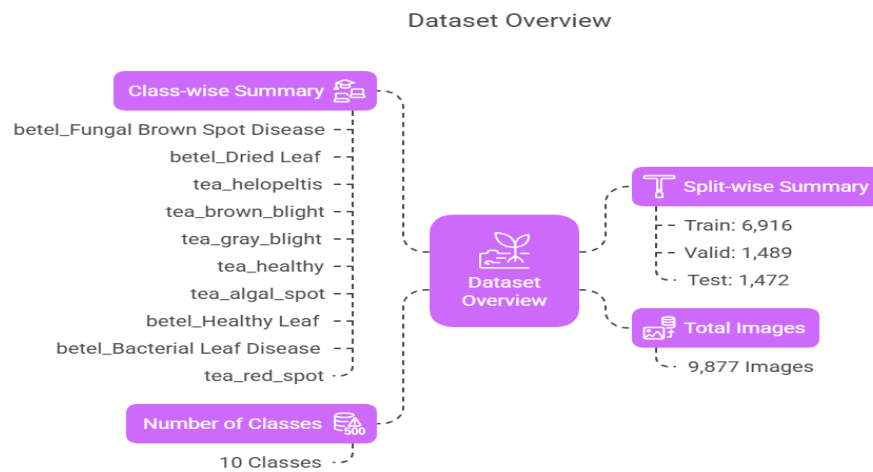


Figure 4: Dataset Overview

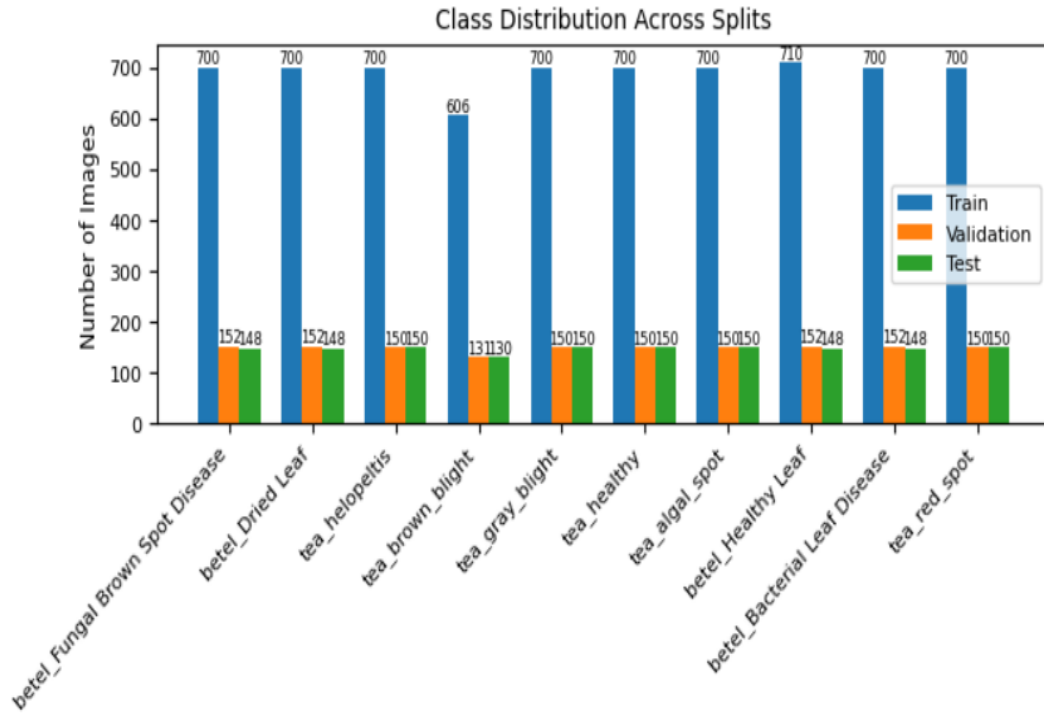


Figure 5: Dataset Distribution

Why it's important:

Splitting the data helps evaluate the model's performance on unseen data. The training set is used to teach the model. The validation set helps monitor the model's performance during training to avoid overfitting. The testing set gives a final, unbiased evaluation of the model's generalization power. A stratified split ensures that all classes are proportionally represented in each subset, maintaining class balance across splits.

Final Dataset Summary

After preprocessing, the unified dataset used for training and evaluation consisted of:

- 10 total classes (6 tea leaf disease types and 4 betel leaf conditions)
- ~1000 images per class (after augmentation)
- Split into training (70%), validation (15%), and testing (15%)

This preprocessing pipeline ensured a balanced, standardized, and bias-free dataset, forming a strong foundation for building accurate and generalizable deep learning models for plant disease classification.

3.3.3 IMAGE RESIZING

All images were resized to 256×256 pixels.

Why it's important:

Neural networks require fixed-size input. Resizing ensures uniformity, allowing images of different dimensions to be processed consistently. The choice of 256×256 strikes a balance between preserving image detail and reducing computational load, which is especially beneficial when training different lightweight models.

3.3.4 NORMALIZATION

Each image was normalized by dividing pixel values by 255 to bring them into the $[0, 1]$ range.

Why it's important:

Most deep learning models converge faster and perform better when inputs are normalized. It ensures numerical stability, faster training, and improves gradient flow, which helps the model learn more efficiently. It also prevents domination of large pixel values during optimization.

3.4 NEURAL NETWORK ARCHITECTURES AND ATTENTION MECHANISMS

3.4.1. TRANSFER LEARNING AND PRE-TRAINED MODELS

Transfer Learning is a powerful technique in deep learning where knowledge gained from solving one problem is leveraged to solve a different but related problem. Instead of training a model from scratch—which can be computationally expensive and requires large amounts of labelled data—transfer learning allows developers to utilize existing models that have already learned rich feature representations from large-scale datasets.

This method is particularly useful when working with limited data, as it helps avoid overfitting and speeds up model convergence. In most transfer learning workflows, the early layers of a neural network (which capture general features like edges, textures, and shapes) are retained, while the later layers (which are more task-specific) are either replaced or fine-tuned for the new problem.

Pre-trained Models are deep learning models that have been trained on massive benchmark datasets, such as ImageNet (over 14 million labeled images across 1,000 categories). These models have already learned to identify low-level and high-level features from diverse image types, making them ideal candidates for transfer learning in domain-specific tasks.

There are two primary ways to use pre-trained models:

- **Feature Extraction:** The pre-trained model acts as a fixed feature extractor. Its layers are frozen, and only the classifier head (fully connected layers) is trained on the new task.
- **Fine-Tuning:** Some or all of the pre-trained model's layers are unfrozen and further trained on the new dataset, allowing it to adapt to task-specific patterns while retaining useful generic features.

MobileNetV3, MnasNet, EfficientNet are some of the examples of popular lightweight pre trained models.

3.4.2 ATTENTION MECHANISMS

Attention mechanisms are techniques that allow convolutional neural networks (CNNs) to dynamically focus on the most informative features in an image, improving learning efficiency and model performance. Especially in tasks like leaf disease classification—where the infected regions might be subtle and surrounded by noise—attention helps the model concentrate on disease-specific patterns while ignoring irrelevant background features.

In CNNs, attention can be applied across various dimensions:

- Channel-wise Attention (e.g., SE – Squeeze-and-Excitation): Focuses on *what* features are important by reweighting the feature channels. This helps the network prioritize channels that carry more disease-relevant information.
- Spatial Attention (e.g., CA – Coordinate Attention): Focuses on *where* the most important features are located within the spatial domain of the image. CA captures directional information along the height and width to highlight crucial leaf regions.
- Layer-level Attention (e.g., ELA – Efficient Local Attention): Operates across network depths to assign weights to different layers. It ensures the model effectively aggregates features from shallow to deep layers, maintaining a balance between low-level texture and high-level semantic information.

3.4.2.1. SQUEEZE-AND-EXCITATION (SE)

The Squeeze-and-Excitation (SE) block is a channel-wise attention mechanism introduced to enhance the representational power of convolutional neural networks by explicitly modeling interdependencies between channels.

How SE Blocks Work:

1. Squeeze (Global Information Embedding):

- A global average pooling operation condenses each feature map (channel) into a single numerical value.
- This “squeezes” the spatial dimensions, summarizing the global distribution of responses in each channel.

2. Excitation (Channel Reweighting):

- The pooled values pass through a small feedforward neural network (typically two fully connected layers with a ReLU in between).
- This network learns nonlinear channel relationships and outputs a set of weights between 0 and 1 for each channel.

3. Recalibration (Attention Application):

- The learned weights are applied to the original feature maps through channel-wise multiplication.
- Channels that are more relevant to the task are amplified, while less important ones are suppressed.

Squeeze-And-Excite (SE) layer

Notes: Tensors' shapes are for illustration purposes. They vary according to the layer's params. The first dimension of tensors' shapes is always the batch size.

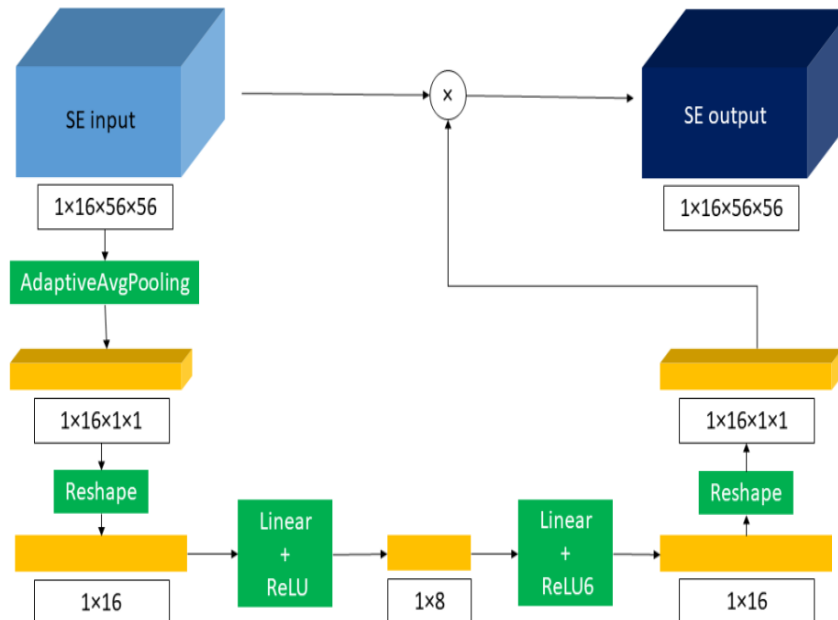


Figure 6: Architecture of Squeeze and Excite Attention Mechanism

Role in MobileNetV3:

- MobileNetV3 uses SE blocks as its default attention mechanism in its bottleneck layers.
- This inclusion enhances feature selection without adding much computational overhead, making it suitable for real-time applications on mobile and embedded devices.
- The integration of SE in MobileNetV3 contributes to its balance between accuracy and efficiency.

Limitations of SE:

- While effective, SE focuses only on *channel-level dependencies* and ignores spatial relationships between features.
- For tasks like leaf disease detection, where *location* matters as much as *what features* are present, this limitation may lead to suboptimal results.

This motivates the exploration of more spatially aware or multi-level attention mechanisms, such as Coordinate Attention (CA) and Efficient Local Attention (ELA), which can better capture spatial and hierarchical information in plant disease images.

3.4.2.2 COORDINATE ATTENTION (CA)

CA enhances both channel and spatial attention by encoding positional information through 1D spatial encoding along height and width.

- Captures where and what to focus on.
- Improves localization of key features in images.

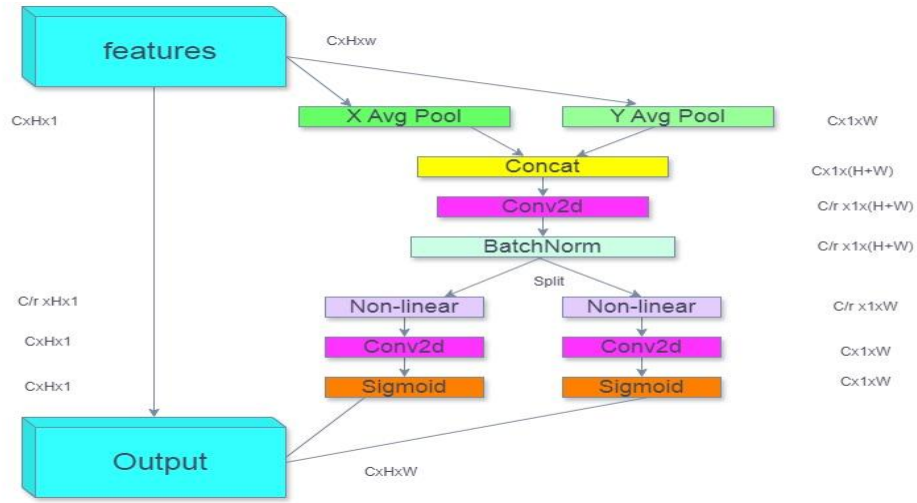


Figure 7: Architecture of Coordinate Attention (CA)

3.4.2.3 EFFICIENT LOCAL ATTENTION (ELA)

ELA is a lightweight module that emphasizes important **layer-level features** by dynamically adjusting their influence during training.

- Reduces redundancy and improves generalization.
- Performs well on small, imbalanced, and augmented datasets.

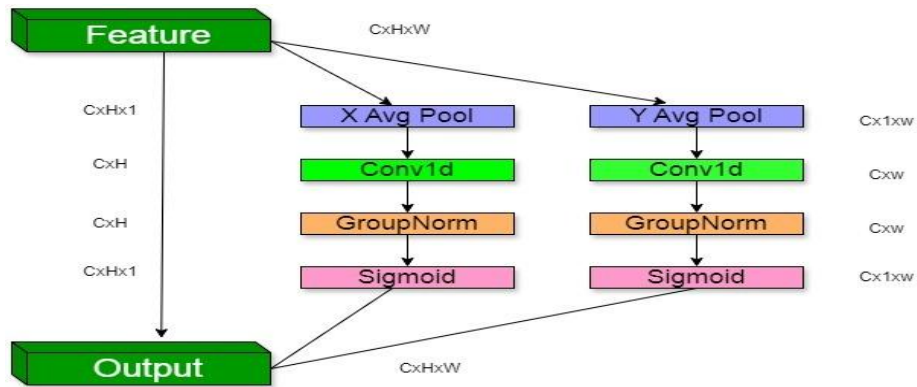


Figure 8: Architecture of Efficient Local Attention (ELA)

3.4.3. MOBILENET V3

MobileNetV3 is a state-of-the-art lightweight convolutional neural network (CNN) architecture specifically designed for mobile and embedded vision applications. Developed by Google, it is an advancement over its predecessors (MobileNetV1 and V2), offering a better trade-off between speed, accuracy, and model size.

Key Features of MobileNetV3:

- **Efficient Architecture:** Combines techniques like depthwise separable convolutions (introduced in V1) and inverted residuals with linear bottlenecks (from V2) to reduce the number of computations without sacrificing performance.
- **Neural Architecture Search (NAS):** MobileNetV3's architecture was partially discovered using NAS, which automatically searched for optimal building blocks tailored to mobile hardware constraints.
- **Built-in Attention Mechanism:** Integrates Squeeze-and-Excitation (SE) blocks by default to improve the model's focus on informative feature channels.

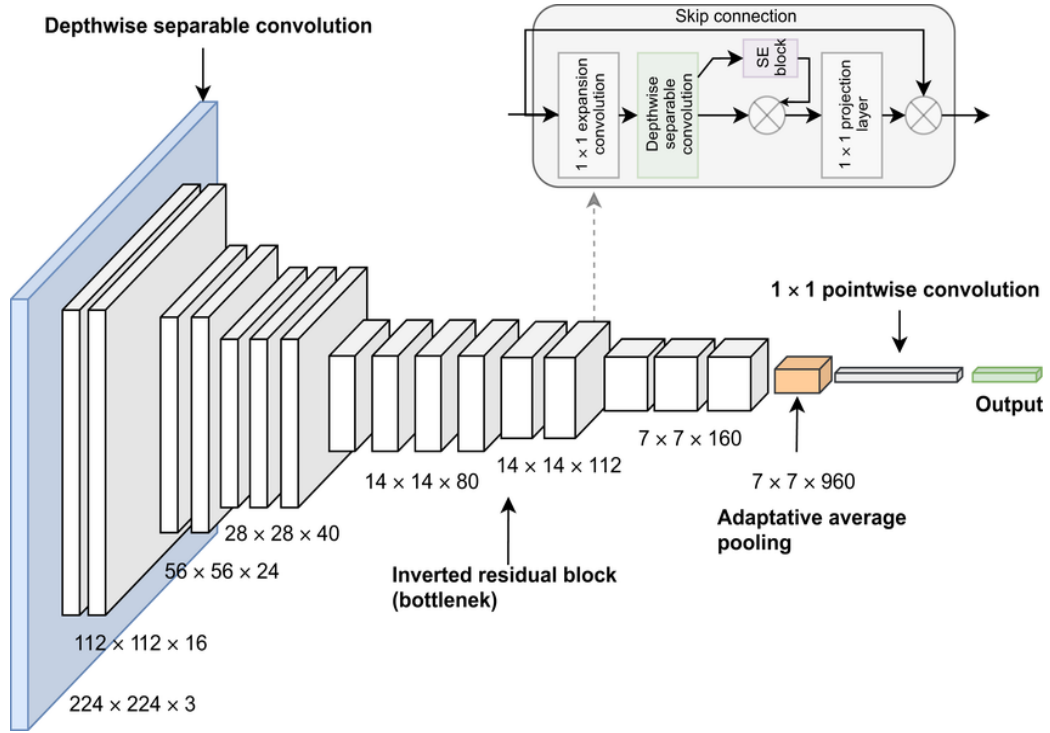


Figure 9: Architecture Diagram of MobileNetV3

Two Variants:

- MobileNetV3-Large: Designed for high-performance tasks (e.g., object detection, classification).
- MobileNetV3-Small: Optimized for low-resource scenarios where power and speed are critical.

Why Use MobileNetV3 in This Study:

- Its lightweight nature makes it ideal for real-time plant disease detection on mobile devices, especially beneficial for smallholder farmers with limited access to high-end computing resources.
- The architecture is modular and adaptable, making it suitable for integrating custom attention mechanisms like Coordinate Attention (CA) and Efficient Local Attention (ELA) for enhanced performance in leaf disease classification tasks.

Table 2: Mobilenet V3 Architecture

Input	Operation	SE Module	Activation Function	Stride
224×224×3	Conv2d	–	HS	2
112×112×16	Bneck, 3×3	–	RE	1
112×112×16	Bneck, 3×3	–	RE	2
56×56×24	Bneck, 3×3	–	RE	1
56×56×24	Bneck, 5×5	√	RE	2
28×28×40	Bneck, 5×5	√	RE	1
28×28×40	Bneck, 5×5	√	RE	1
28×28×40	Bneck, 3×3	–	HS	2
14×14×80	Bneck, 3×3	–	HS	1
14×14×80	Bneck, 3×3	–	HS	1
14×14×80	Bneck, 3×3	–	HS	1
14×14×80	Bneck, 3×3	√	HS	1
14×14×112	Bneck, 3×3	√	HS	1
14×14×112	Bneck, 5×5	√	HS	2
7×7×160	Bneck, 5×5	√	HS	1
7×7×160	Bneck, 5×5	√	HS	1
7×7×160	Conv2d, 1×1	–	HS	1
7×7×960	Pool, 7×7	–	–	1
1×1×960	Conv2d, 1×1, NBN	–	HS	1
1×1×1280	Conv2d, 1×1, NBN	–	–	1

Table 2 presents detailed layer-wise configuration of the MobileNetV3 model, designed for high-performance tasks with efficiency in mind. The table outlines the input dimensions, operations applied at each layer, whether the Squeeze-and-Excitation (SE) attention module is used, the type of activation function, and the stride for spatial dimensionality reduction.

3.4.4 PROPOSED MODEL: MOBILENET V3 + ELA

In this study, we propose a modified convolutional neural network architecture by integrating Efficient Local Attention (ELA) into MobileNetV3, aiming to enhance the model’s feature extraction capabilities while maintaining computational efficiency. MobileNetV3 is a lightweight and optimized deep learning model designed specifically for mobile and embedded applications. It employs depthwise separable convolutions, inverted residual blocks, and the Squeeze-and-Excitation (SE) attention mechanism to improve performance with reduced parameter overhead.

However, the SE blocks used in MobileNetV3 primarily focus on channel-wise attention and do not fully exploit the contextual relationships across different network layers. To address this limitation, we substitute the SE blocks with Efficient Local Attention (ELA) modules. ELA provides layer-wise attention by learning to weigh the contribution of features from different layers dynamically, thereby enabling more robust hierarchical feature aggregation. This is particularly beneficial in tasks like leaf disease detection, where both local texture and global shape patterns are important.

The architectural flow of the proposed MobileNetV3 + ELA model follows the base structure of MobileNetV3, replacing the SE attention units with ELA modules at corresponding bottleneck blocks. The model is trained using a two-phase transfer learning approach:

- Phase 1 (Baseline Training): All pre-trained layers are frozen, and only the final classification layers along with the attention module are trained.
- Phase 2 (Fine-tuning): The last 20 layers of the backbone are unfrozen to allow fine-grained learning specific to the leaf disease classification task.

This design ensures the model remains lightweight and deployable on resource-constrained devices, such as smartphones or edge processors, while still achieving high classification

performance. The inclusion of ELA enhances the model's ability to focus on discriminative features across various depths of the network, making it more robust to variations in lighting, leaf shape, and disease manifestation.

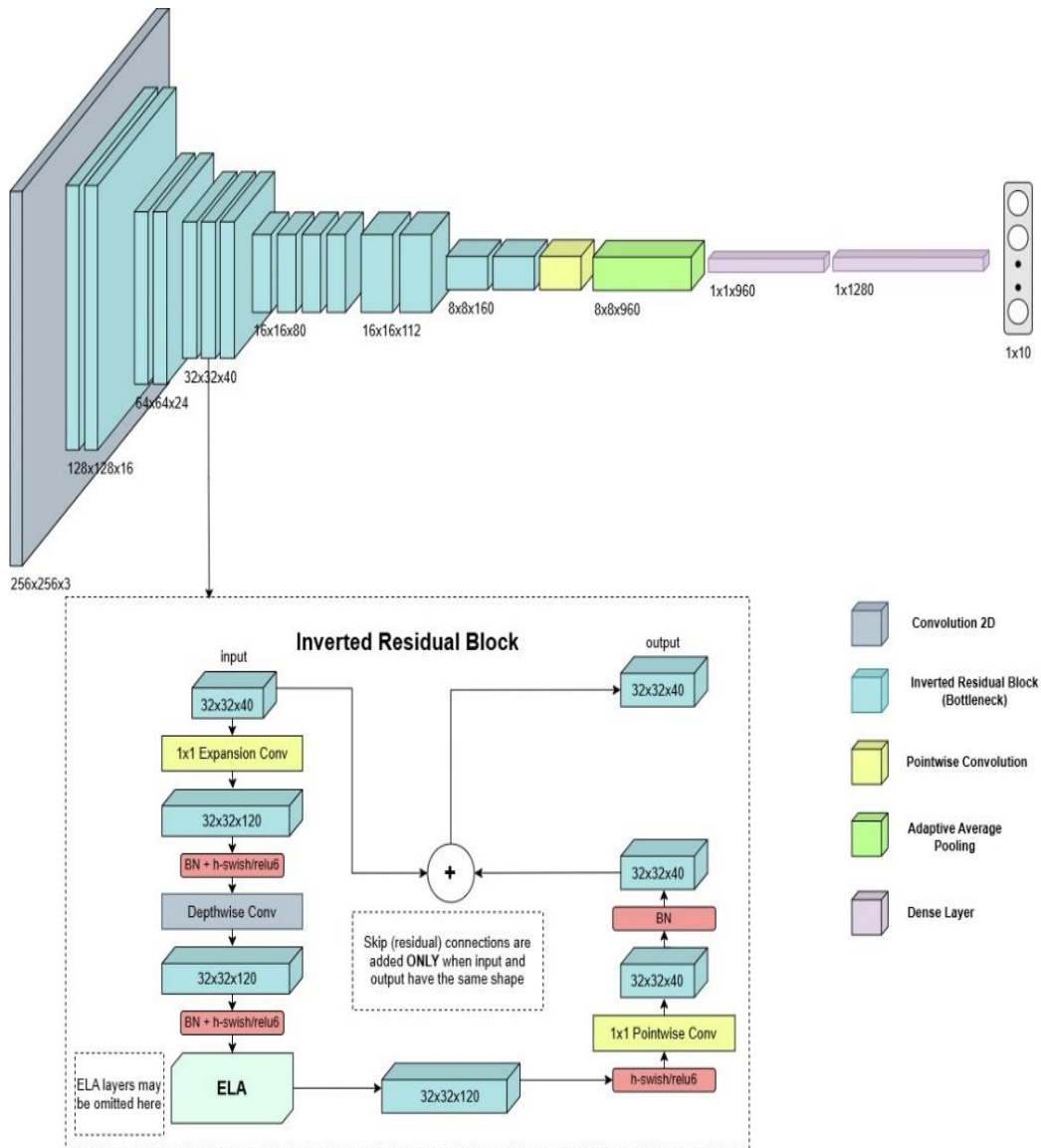


Figure 10: Proposed Model (MobileNetV3 with ELA)

Motivation Behind Attention Module Replacement in MobileNetV3

Recent studies on large-scale datasets such as ImageNet have demonstrated that advanced attention mechanisms like Coordinate Attention (CA) and Efficient Local Attention (ELA) consistently outperform traditional Squeeze-and-Excitation (SE) blocks in terms of both accuracy and generalization. Motivated by these findings, we modified the baseline MobileNetV3 model by replacing its default SE blocks with CA and ELA modules in separate experimental setups. This adaptation aims to explore whether these state-of-the-art attention mechanisms—proven effective on large datasets—can also enhance performance on our custom dataset comprising tea and betel leaf diseases. By conducting this comparison, we seek to identify the most effective attention strategy for improving classification in lightweight models suitable for real-world deployment.

Chapter 4

EXPERIMENTAL RESULTS AND ANALYSIS

4.1 TEST ENVIRONMENT AND PARAMETER SETTING

To evaluate the performance of the tea and betel leaf diseases recognition model MobileNetV3-ELA, acquired images of tea and betel leaf diseases were used for both training and testing. The dataset was divided into training, validation, and test sets in a ratio as aforementioned. The experiment was employed the PyTorch deep learning framework and was programmed in Python. The development environment was set up using Google Colaboratory Jupyter Notebook with the runtime type selected as T4 GPU.

The Batch Size of the experiment was set to 16. In order to improve the convergence of the model, the classified cross-entropy is used as the loss function to train the model. The optimizer was chosen to be Adaptive Moment Estimation (Adam) with a learning rate of 0.0003.

Table 3: Parameter Setting

Sl.No.	Setting	Value	Description
1	Batch Size	16	Number of samples per training batch.
2	Optimizer	Adam	Adaptive Moment Estimation optimization algorithm.
2	Learning rate	0.0003	Step size for model weight updates during training.
4	Epochs	40	40 epochs per training phase.
5	Loss Function	Cross Entropy Loss	Used for multi-class classification tasks.

4.2 EVALUATION INDEX

In order to comprehensively evaluate the performance of the MobileNetV3-ELA model, this experiment selected four indicators to comprehensively evaluate the recognition effect of the model: Precision, Recall, F1-score and Accuracy. The calculation formulas are as follows:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

In the formula, TP, FP, FN and TN are the statistics of the classification of different tea and betel leaf diseases by the classification model in the confusion matrix respectively. Among them, TP (True Positive) represents the number of samples whose true value is positive and identified as positive, FP (False Positive) represents the number of samples whose true value is negative but identified as positive, FN (False Negative) represents the number of samples whose true value is positive but identified as negative, and TN (True Negative) represents the number of samples whose true value is negative and identified as negative. For the purpose of identifying diseases, the actual number of categories of samples to be identified is regarded as the positive sample number, while the sum of all other categories is considered the negative sample number.

4.3 COMPARATIVE EXPERIMENT OF TRANSFER LEARNING TRAINING METHODS

There are three common ways of transfer learning: the full parameter migration method, which involves freezing all convolution layers and only training the fully connected layer; the reuse model method, which only uses the model structure but not the pre-trained parameters; and the fine-tuning method, which involves freezing part of the convolution layers. This study adopts the fine-tuning method of freezing only part of the convolution layers. The last 20 layers of the architecture were unfreeze to apply the fine-tuning method. To test the effectiveness of the fine-tuning method used in this study, the full migration and the fine-tuning transfer learning methods were used to train two models. Experiments were conducted based on the improved MobileNetV3-ELA model using the same experimental data. The experimental results are shown in Table 3, and the variation curve of training and validation accuracy of the three transfer learning methods is shown in Figure 10.

Table 4: Performance of MobileNetV3-ELA trained by 2 different transfer methods

Methods	Precision (%)	Recall (%)	F-1 Score (%)	Accuracy (%)
Full-migration	97.25	97.28	97.23	97.28
Fine-tuning	98.47	98.50	98.48	98.51

It can be seen from Table 3 that under the same experimental conditions, the accuracy, precision, recall and the F-1 score of the fine-tuning method (unfreezing last 20 layers) is higher than that of the full migration method.

In the full parameter migration method, all the initial parameters of the model are obtained by pre training, and only the fully connected layer is trained, making it difficult to optimize the model's parameters.

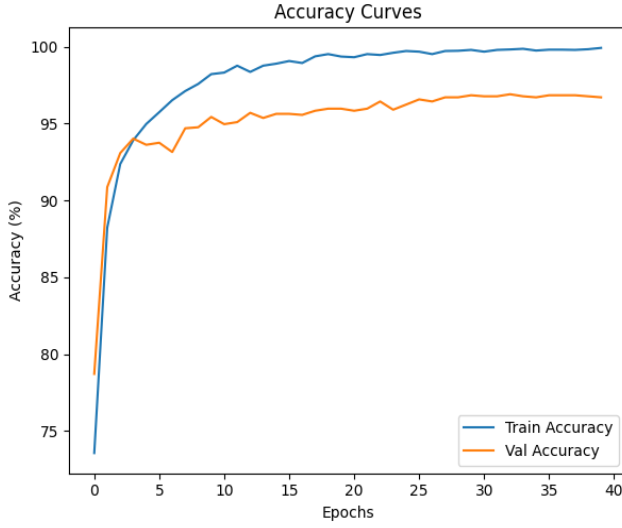


Figure 11: Accuracy curves of full migration method

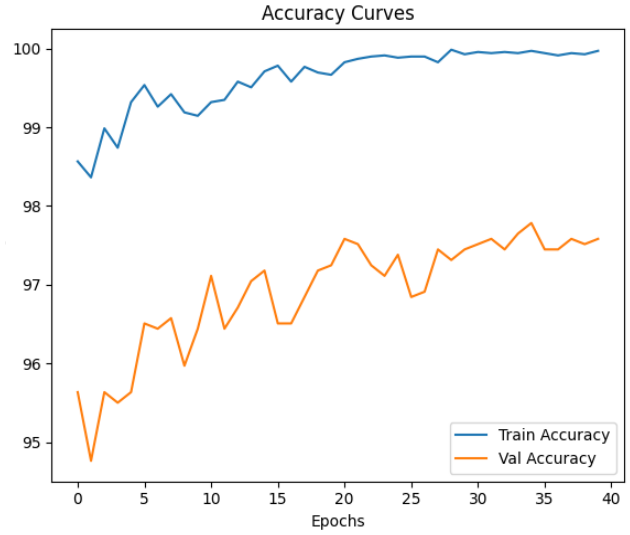


Figure 12: Accuracy curves of fine-tuning method

As shown in Figure 10, the fine-tuning method uses initial parameters obtained from extensive data training, enabling the model to converge more quickly to suitable parameters. Additionally, this method retraines the convolution layers in the middle of the model, making the parameters more tailored to the tea and betel leaf disease identification task. As a result, the four indicators in the comprehensive experiment demonstrate that the fine-tuning transfer learning method used in this study is more effective than the full migration transfer learning method.

4.4 PERFORMANCE ANALYSIS OF MOBILENET V3 ELA MODEL

In the fine-tuning transfer learning method, the loss value change curve of the MobileNetV3-ELA model on our self-built training set (Figure 11) exhibits three distinct phases. During the first ten epochs, training loss plunges sharply from 0.0431 (Epoch 1) to 0.0238 (Epoch 10), while validation loss likewise falls from 0.2158 to 0.2162, and accuracy climbs from 95.63 % to 96.44 %. Between Epochs 11 and 20, the rate of decrease slows: by Epoch 20, training loss reaches approximately 0.0082 and validation loss settles at 0.1579, with validation accuracy improving to 97.25 %. After Epoch 20, both curves begin to flatten—as seen in the final epochs (e.g., Epoch 40 train loss 0.0006, val loss 0.1700)—indicating the model has approached saturation. Crucially, the near-parallel trajectories of the training and validation loss

throughout all 40 epochs demonstrate consistent convergence without divergence, confirming that MobileNetV3-ELA learns effectively from our data without overfitting.

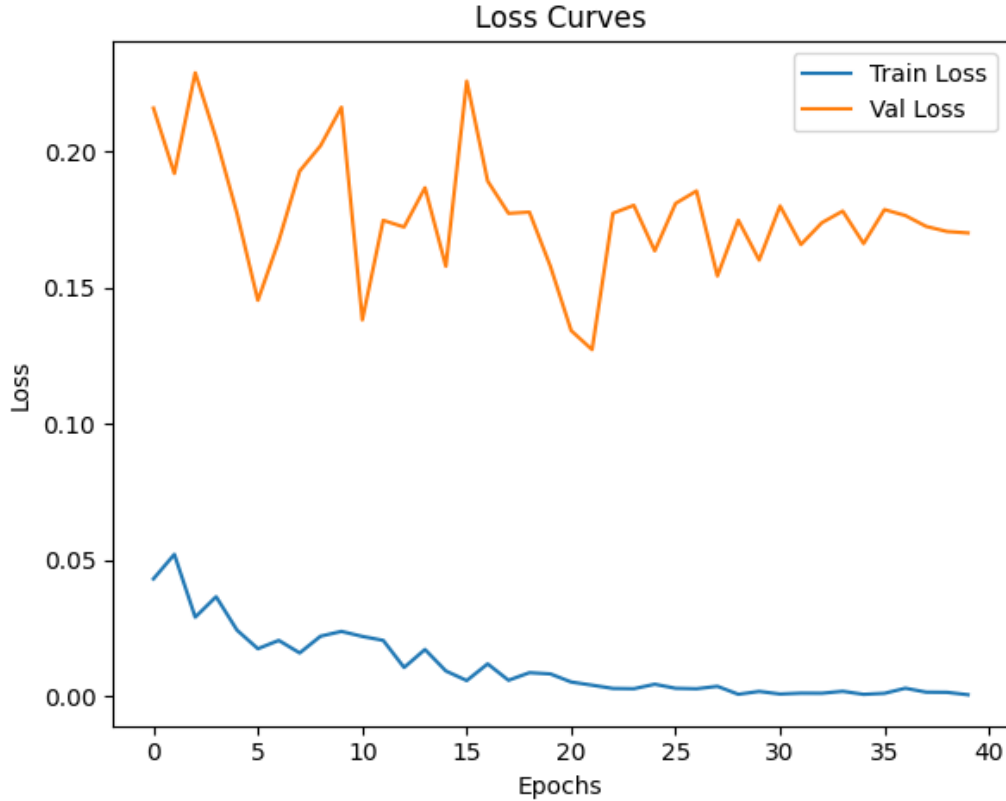


Figure 13: Loss Value curves of the proposed model

In order to further verify the performance of the MobileNetV3-ELA model, the classification results on the self-built test set were analyzed. The test set contained 10 combined different classes of tea and betel leaf diseases, including around 150 images for each class, for a total of 1472 images. Overall, the average recognition accuracy, recall rate, and F1-score of the MobileNetV3-ELA model on the test set are 98.51%, 98.50%, and 98.48%, respectively, all exceeding 98%. These experimental results demonstrate that the improved MobileNetV3-ELA model can efficiently locate and extract small feature differences in tea and betel leaf disease images.

4.4.1 CONFUSION MATRIX

Figure 12 shows the confusion matrix of the proposed model MobileNet V3-ELA on the combined betel and tea leaf test set. Each row corresponds to the true class and each column to the model's prediction. The strong concentration of counts along the main diagonal-ranging from 128 to 150 correct predictions per class-demonstrates that the model reliably distinguishes among all ten disease and healthy-leaf categories.

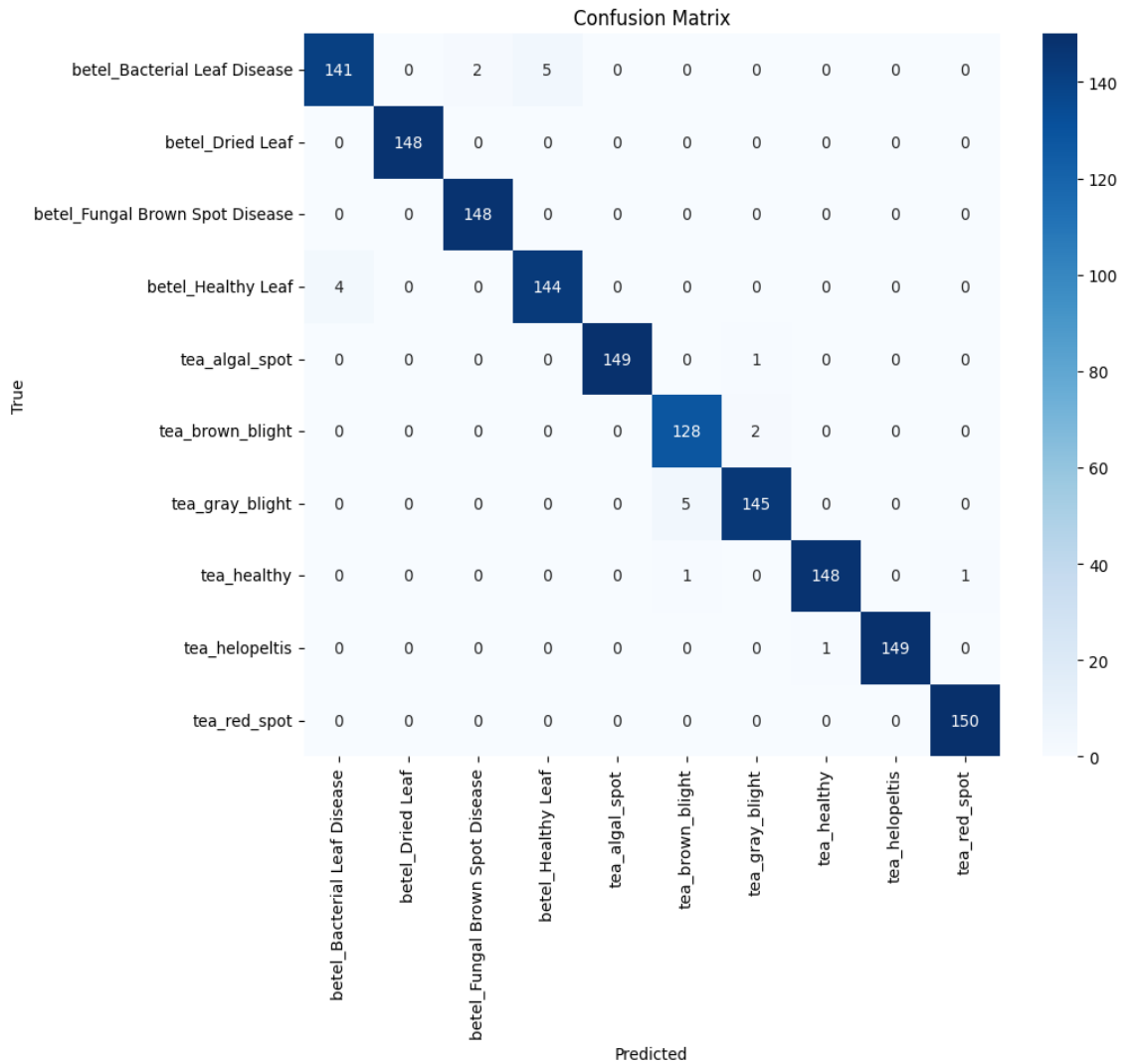


Figure 14: Confusion Matrix of the proposed model

For the betel leaf subset, the model correctly identifies 141 out of 148 instances of Bacterial Leaf Disease, with the remaining 7 misclassified mainly as Healthy Leaf and a small fraction

as Fungal Brown Spot Disease. Both Dried Leaf and Fungal Brown Spot Disease are perfectly classified, achieving 148 correct predictions each. Healthy betel leaves are recognized with 144 out of 148 accurate identifications, while 4 are mistakenly labeled as Bacterial Leaf Disease.

Within the tea leaf categories, Algal Spot is detected with 149 correct predictions out of 150, with a single case confused as Brown Blight. Brown Blight itself yields 128 correct identifications out of 130, with 2 samples misclassified as Gray Blight. Gray Blight achieves 145 accurate predictions out of 150, with five instances predicted as Brown Blight. Healthy tea leaves are classified correctly in 148 of 150 cases, although 1 each is mislabeled as Brown Blight and Red Spot. Helopeltis shows similarly strong performance, with 149 of 150 correct predictions and 1 false Healthy classification. Finally, Red Spot reaches perfect accuracy, with all 150 samples correctly recognized.

Overall, the few misclassifications occur primarily between visually similar symptom classes—such as Brown versus Gray Blight or Healthy versus Bacterial leaf conditions—underscoring the model’s high accuracy and robustness when discriminating among closely related leaf-disease categories.

4.4.2 CLASSIFICATION REPORT

Table 4 presents the classification report of the proposed model (MobileNet V3-ELA), which summarizes the precision, recall, and F1-score metrics for each class in the combined betel and tea leaf dataset. The performance across all categories is consistently high, highlighting the model’s ability to accurately differentiate between various diseases and healthy leaf conditions.

Table 5: *Classification Report of the proposed model*

Class Labels	Precision (%)	Recall (%)	F-1 Score (%)
betel bacterial leaf	97.24	95.27	96.25
betel dried leaf	100	100	100
betel fungal brown spot	98.67	100	99.33
Betel healthy leaf	96.64	97.30	96.97
tea algal spot	100	99.33	99.67
tea brown blight	95.52	98.46	96.97
tea gray blight	97.97	96.67	97.32
tea healthy	99.33	98.67	99.00
tea helopeltis	100	99.33	99.67
tea red spot	99.34	100	99.67
Accuracy (%)	98.51		
Macro Average (%)	98.47	98.50	98.48

For the betel leaf categories, the model achieves perfect precision and recall (100%) for both the Dried Leaf and Fungal Brown Spot classes, resulting in an F1-score of 100% and 99.33%, respectively. The Bacterial Leaf class attains a precision of 97.24%, recall of 95.27%, and an F1-score of 96.25%. Healthy betel leaves are classified with a precision of 96.64%, recall of 97.30%, and an F1-score of 96.97%, demonstrating strong but slightly lower performance compared to the diseased categories.

In the tea leaf segment, the model continues to show exceptional performance. Tea Algal Spot and Tea Helopeltis achieve perfect precision (100%) and near-perfect recall (99.33%), both yielding an F1-score of 99.67%. Tea Red Spot also reaches near-perfect metrics with 99.34% precision, 100% recall, and 99.67% F1-score. Tea Brown Blight, while slightly less precise, still scores high with 95.52% precision, 98.46% recall, and 96.97% F1-score. Tea Gray Blight and Tea Healthy show strong results as well, with precision above 97% and F1-scores above 97%.

The overall accuracy of the model is 98.51%, while the macro-averaged precision, recall, and F1-score stand at 98.47%, 98.50%, and 98.48%, respectively. These results confirm the

robustness and generalization capability of MobileNet V3-ELA across multiple leaf conditions, making it a reliable solution for automated leaf disease detection.

4.5 COMPARATIVE TEST OF DIFFERENT MODELS

Table 5 presents a comprehensive comparison of five convolutional network architectures, Simple CNN, M-NasNet A1, MobileNet V3, MobileNet V3 + CA (Coordinate Attention module), and our proposed MobileNet V3 + ELA (Efficient Local Attention module), all evaluated on the same tea-and-betel leaf test set under identical training conditions. The metrics reported include overall accuracy, total number of learnable parameters, precision, recall, and F1-score.

Table 6: Comparison of accuracy, parameters and computation between models

Models	Accuracy (%)	Params (M)	Precision (%)	Recall (%)	F-1 Score (%)
Simple CNN	79.62	228,632	75.18	79.47	76.31
M-Nasnet A1	97.69	3,115,122	97.72	97.71	97.65
MobileNet V3	97.49	4,214,842	97.46	97.50	97.46
MobileNet V3 + CA	97.62	2,997,657	97.61	97.64	97.62
MobileNet V3 + ELA (Ours)	98.51	2,872,914	98.47	98.50	98.48

Our MobileNet V3 + ELA model attains the highest test accuracy of **98.51 %**, outperforming MobileNet V3 + CA (97.62 %), the vanilla MobileNet V3 backbone (97.49 %), and M-NasNet A1 (97.69 %). This 0.89 percentage-point gain over the CA-enhanced variant and

a 1.02-point gain over the base backbone demonstrates the strong discriminative power introduced by the Efficient Local Attention module, which adaptively emphasizes salient regions in the feature maps without significant computational overhead.

Remarkably, these gains are achieved with **fewer parameters**. MobileNet V3 + ELA contains **2,872,914** parameters, which is **1,341,928 fewer** than MobileNet V3's **4,214,842**, and **124,743 fewer** than MobileNet V3 + CA's **2,997,657**. This represents parameter reductions of approximately **32 %** and **4 %**, respectively, highlighting how ELA not only improves accuracy but also streamlines the model for deployment on resource-constrained devices.

In terms of precision, recall, and F1-score, the proposed model consistently leads with **98.47 % precision, 98.50 % recall, and 98.48 % F1-score**. By contrast, MobileNet V3 + CA achieves 97.61 % precision, 97.64 % recall, and 97.62 % F1-score, while the base MobileNet V3 model records 97.46 % precision, 97.50 % recall, and 97.46 % F1-score. Even M-NasNet A1 with over 3.1 million parameters yields lower F1-score (97.65 %) than our leaner architecture.

Overall, the integration of Efficient Local Attention into the MobileNet V3 backbone delivers the best balance of **high accuracy, compact size, and computational efficiency**, making MobileNet V3 + ELA an ideal solution for on-device leaf disease and pest recognition.

4.6 MAKING PREDICTION FROM THE DATASET

4.6.1 CODE BASED PREDICTIONS

To evaluate the real-world effectiveness of the proposed model, a composite test image containing eight different leaf samples—four tea leaves and four betel leaves—was used. The model successfully identified and classified all eight leaves correctly, demonstrating its strong multi-class recognition capabilities even in a single, cluttered frame. Furthermore, the Grad-CAM visualization of the same image highlights the regions of interest used by the model during prediction. The heatmaps confirm that the model focuses precisely on the diseased areas of each leaf, validating both the accuracy and interpretability of the predictions. This demonstrates the model's potential for real-time deployment in agricultural settings where multiple crops may coexist.

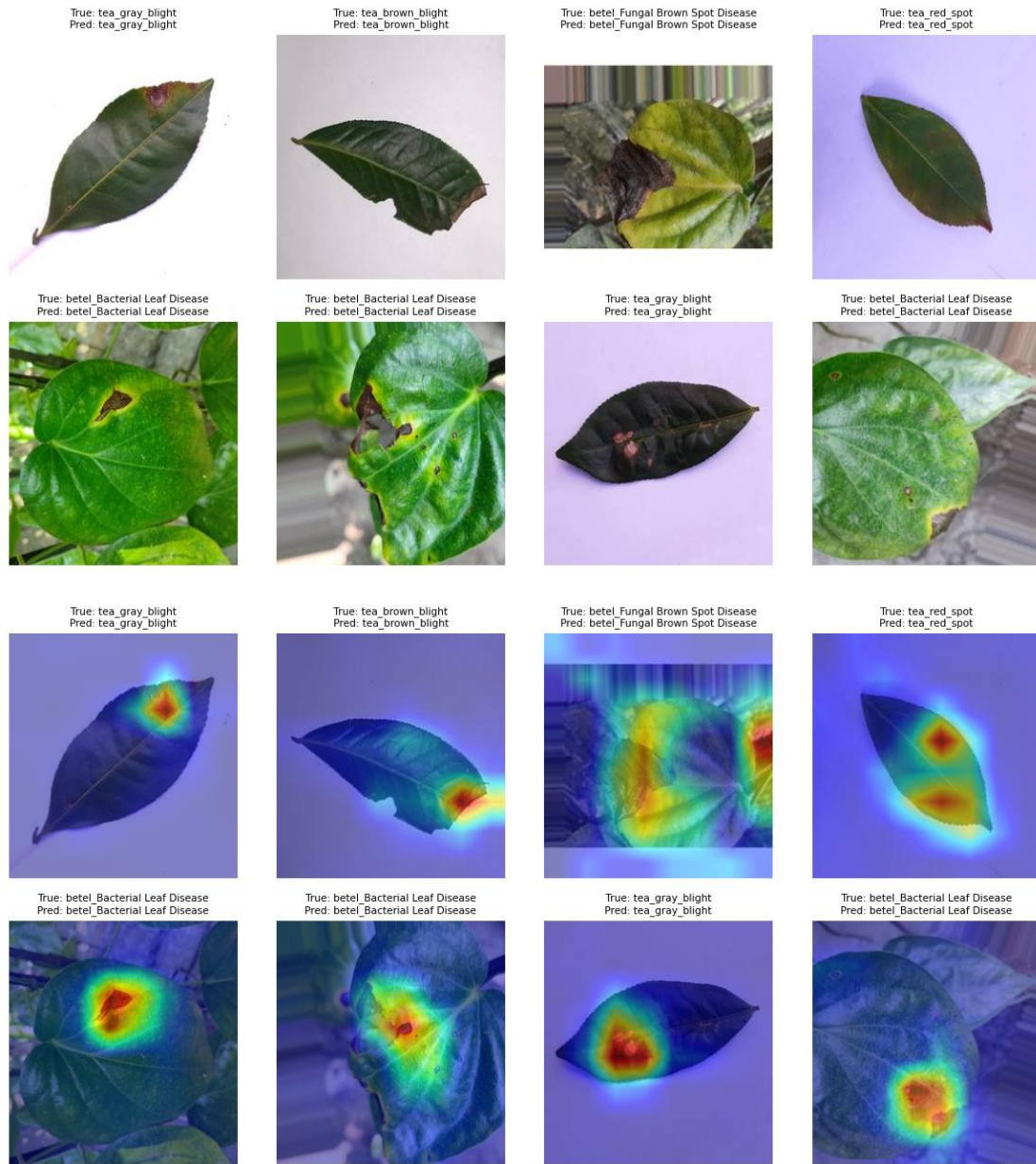


Figure 15: Few predicted as well as grad-cam visualization images by the model

The prediction results demonstrate the reliability and precision of the trained model on the tea and betel leaf test set. Its consistent classification of disease-affected leaves supports its potential application in automated disease detection systems in agricultural domains.

4.6.2 WEB APP BASED PREDICTION

The developed web application serves as an intuitive and interactive platform for predicting diseases in **tea and betel leaves** using a deep learning model. Built upon a lightweight and efficient architecture (MobileNetV3 with Efficient Layer Attention), the app allows users to upload leaf images and instantly receive disease predictions along with confidence scores.

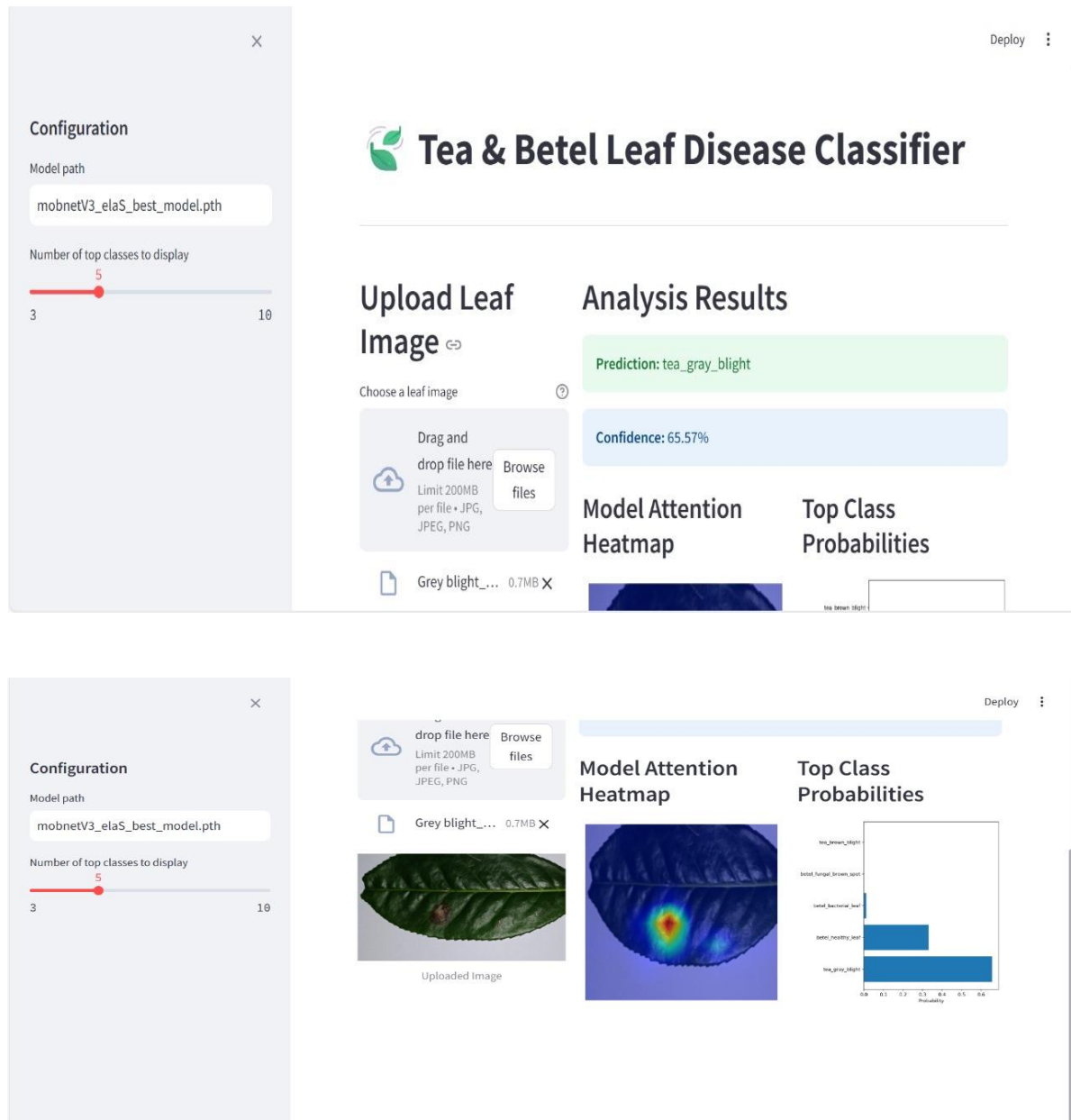


Figure 16: Snapshots of web app-based prediction

Once an image is uploaded, the model processes it to identify the correct disease class, displaying the top predicted label, confidence percentage, and a bar chart of class probabilities. To enhance transparency and interpretability, the app also provides Grad-CAM visualizations, highlighting the regions of the leaf that influenced the model's decision. This is particularly valuable for gaining insights into model behaviour and ensuring trustworthiness in prediction.

Chapter 5

CODE IMPLEMENTATION

5.1 IMPORTING LIBRARIES

```
import math
from functools import partial
from typing import Any, Callable, List, Optional, Sequence

import torch
from torch import nn, Tensor
from torchvision.models.mobilenetv3 import (
    MobileNetV3,
    InvertedResidualConfig,
    _mobilenet_v3_conf,
    _mobilenet_v3,
)
```

```
import os
import zipfile
import time
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import CosineAnnealingLR
from torch.utils.data import DataLoader
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
from google.colab import drive
```

5.2 DATA PREPROCESSING: AUGMENTATION OF BETEL LEAVES

```
# Create class directories in augmented folder
classes = os.listdir(original_dir)
for class_name in classes:
    class_path = os.path.join(original_dir, class_name)
    if os.path.isdir(class_path):
        augmented_class_path = os.path.join(augmented_dir, class_name)
        os.makedirs(augmented_class_path, exist_ok=True)

    # Process each image
    for image_file in os.listdir(class_path):
        image_path = os.path.join(class_path, image_file)

        # Skip non-image files
        ext = os.path.splitext(image_file)[1].lower()
        if ext not in ['.jpg', '.jpeg', '.png', '.bmp', '.gif']:
            continue

        if os.path.isfile(image_path):
            # Load image and convert to RGB
            img = Image.open(image_path).convert('RGB')
            img_array = np.array(img)

            # Generate 3 augmented images
            for i in range(1, 4):
                augmented_img = datagen.random_transform(img_array)
                augmented_img_pil = Image.fromarray(augmented_img.astype('uint8'))

                # Create new filename
                base, ext = os.path.splitext(image_file)
                new_filename = f"{base}_aug_{i}{ext}"
                new_filepath = os.path.join(augmented_class_path, new_filename)

                # Save augmented image
                augmented_img_pil.save(new_filepath)

print("Data augmentation completed successfully!")
```

5.3 COMBINING TEA AND BETEL DATA INTO ONE

```
# --- 2. Define Paths ---
# *Update these paths to the actual locations of your ZIPPED datasets in Google Drive*
tea_zip_path = '/content/drive/MyDrive/My Datasets/Tea/Tea_Leaf_Final_Dataset.zip' # Zipped Tea Leaf dataset
betel_zip_path = '/content/drive/MyDrive/My Datasets/Betel/Betel_Leaf_Final_Dataset (1).zip' # Zipped Betel Leaf dataset
extracted_tea_folder = '/content/extracted_tea'
extracted_betel_folder = '/content/extracted_betel'

# --- Paths within the final COMBINED dataset directory ---
final_combined_dir = '/content/Tea_Betel_Leaf_Final_Dataset'
train_dir = os.path.join(final_combined_dir, 'train')
test_dir = os.path.join(final_combined_dir, 'test')
val_dir = os.path.join(final_combined_dir, 'valid')

# --- Output path for the combined zipped dataset ---
# *** IMPORTANT: This is where the combined dataset will be SAVED on your Google Drive ***
output_zip_path = '/content/drive/MyDrive/Tea_Betel_Leaf_Final_Dataset(1).zip'
```

5.4 RESIZING AND MERGING WITH ACTUAL CLASS HANDLING

```
# =====
# 2. Resize & Merge with Actual Class Handling
# =====
def resize_image(img_file, save_path):
    """Resize with aspect ratio preservation and padding"""
    try:
        img = Image.open(img_file).convert('RGB')
        img.thumbnail(TARGET_SIZE, Image.Resampling.LANCZOS)

        # Create padded image
        new_img = Image.new("RGB", TARGET_SIZE, (255, 255, 255))
        new_img.paste(img, (
            (TARGET_SIZE[0] - img.size[0]) // 2,
            (TARGET_SIZE[1] - img.size[1]) // 2
        ))

        new_img.save(save_path)
        return True
    except Exception as e:
        print(f"Error processing image: {str(e)}")
        return False

def process_zip(zip_path, output_dir):
    """Handles actual class names and directory structures"""
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        for file_info in zip_ref.infolist():
            if file_info.is_dir():
                continue

            # Handle different zip structures
            if "Betel_Leaf_Raw" in file_info.filename:
                # Raw zip structure: Betel_Leaf_Raw/Class Name/image.jpg
                parts = file_info.filename.split('/')
                if len(parts) < 3:
                    continue # Skip root files
                class_name = parts[1]
```

```

else:
    # Augmented zip structure: Class Name/image_aug_X.jpg
    parts = file_info.filename.split('/')
    if len(parts) < 2:
        continue
    class_name = parts[0]
    file_name = parts[-1]

    # Create class directory with actual name
    class_dir = os.path.join(output_dir, class_name)
    os.makedirs(class_dir, exist_ok=True)

    # Process and save image
    save_path = os.path.join(class_dir, file_name)
    if not os.path.exists(save_path):
        with zip_ref.open(file_info) as f:
            if not resize_image(f, save_path):
                print(f"Failed to process: {file_info.filename}")

def merge_and_resize():
    """Main merge function with actual class names"""
    print("🔧 Starting merge and resize...")

    # Clean directories
    shutil.rmtree(MERGED_DIR, ignore_errors=True)
    os.makedirs(MERGED_DIR, exist_ok=True)

    # Process both zips
    for zip_path in [RAW_ZIP, AUG_ZIP]:
        print(f"🔧 Processing {os.path.basename(zip_path)}...")
        process_zip(zip_path, MERGED_DIR)

    # Verification
    print("\n✅ Merge complete. Class counts:")
    for class_name in sorted(os.listdir(MERGED_DIR)):
        class_path = os.path.join(MERGED_DIR, class_name)
        count = len(os.listdir(class_path))
        print(f"    - {class_name}: {count} images")

```

5.5 OBSERVING SOME OF THE SAMPLE IMAGES [ONE IMAGE FROM EACH CLASS]

```

# Configuration
train_dir = "/content/drive/MyDrive/final_data/train"
class_names = sorted(os.listdir(train_dir))[:10] # First 10 classes
cols = 5 # Images per row
rows = 2 # Number of rows (10/cols)

# Style settings
plt.figure(figsize=(20, 8))
plt.suptitle('Class Representatives', fontsize=16, y=1.05)

for i, class_name in enumerate(class_names):
    # Get first image from each class
    class_dir = os.path.join(train_dir, class_name)
    img_file = sorted(os.listdir(class_dir))[0] # First image
    img_path = os.path.join(class_dir, img_file)

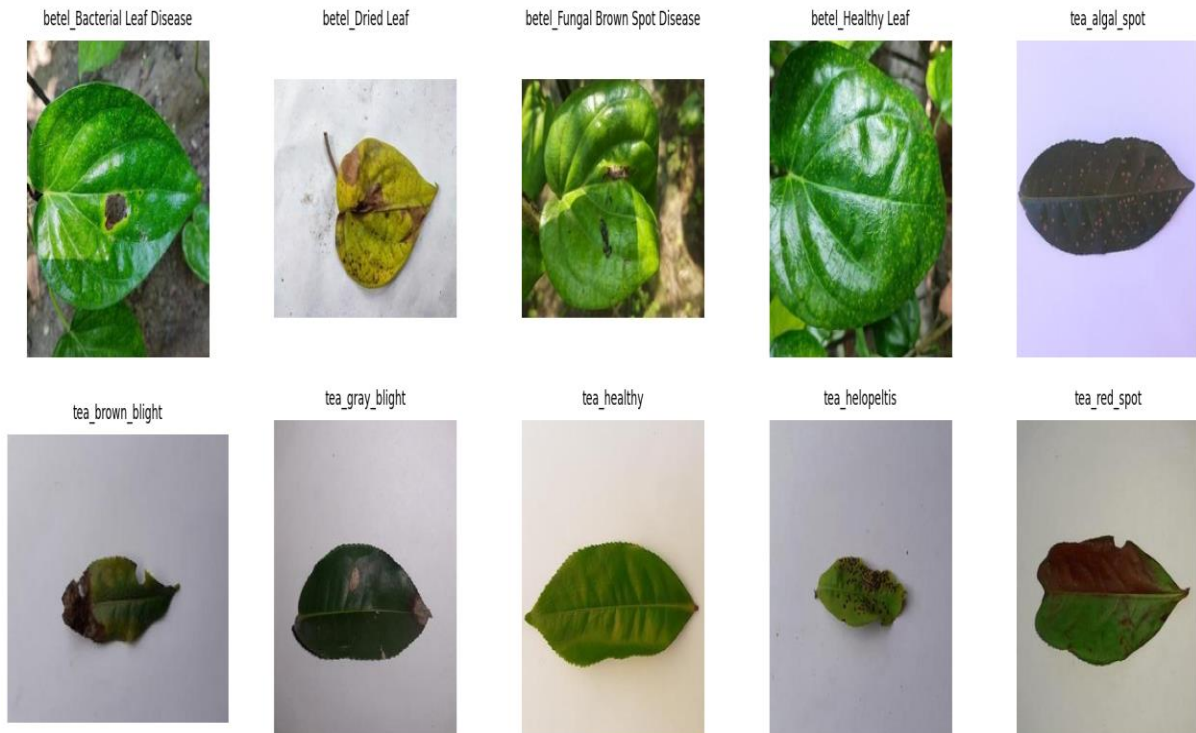
    # Create subplot
    ax = plt.subplot(rows, cols, i+1)
    img = Image.open(img_path)

    # Display with clean formatting
    plt.imshow(img)
    plt.title(class_name, fontsize=12, pad=10)
    plt.axis('off')

    # Add subtle border
    ax.spines['top'].set_visible(True)
    ax.spines['right'].set_visible(True)
    ax.spines['bottom'].set_visible(True)
    ax.spines['left'].set_visible(True)
    ax.spines[:].set_color("#eeeeee")
    ax.spines[:].set_linewidth(1)

```

Class Representatives



5.6 DEFINING ELA

```
#####
# 1. Define the ELA-S Module (Efficient Local Attention - Small Variant)
#####
class ELA_S(nn.Module):
    """
    Efficient Local Attention - Small (ELA-S) module.

    This variant uses:
    - A 1D convolution with kernel size = 5
    - Group Convolution with groups = in_channels/8 (i.e. a slight grouping instead of full depthwise)
    - GroupNorm with a target of 16 groups (dynamically computed)

    It processes the horizontally and vertically pooled feature vectors,
    then reweights the input feature map.
    """
    def __init__(self, inp, kernel_size=5, target_gn_groups=16):
        super(ELA_S, self).__init__()
        self.pool_h = nn.AdaptiveAvgPool2d((None, 1))
        self.pool_w = nn.AdaptiveAvgPool2d((1, None))
        pad = kernel_size // 2
        # Use group convolution with groups = inp/8
        groups = max(1, inp // 8)
        self.conv = nn.Conv1d(inp, inp, kernel_size=kernel_size,
                               padding=pad, groups=groups, bias=False)
        # Compute the number of groups for GN without changing the channel count.
        gn_groups = compute_groups(inp, target_gn_groups)
        self.gn = nn.GroupNorm(gn_groups, inp)
        self.sigmoid = nn.Sigmoid()
```

```

def forward(self, x: torch.Tensor) -> torch.Tensor:
    b, c, h, w = x.size()
    # Pool along height and width to obtain 1D feature vectors.
    x_h = self.pool_h(x).view(b, c, h)
    x_w = self.pool_w(x).view(b, c, w)
    # Apply 1D convolution, GroupNorm, and sigmoid activation.
    a_h = self.conv(x_h)
    a_h = self.gn(a_h)
    a_h = self.sigmoid(a_h).view(b, c, h, 1)
    a_w = self.conv(x_w)
    a_w = self.gn(a_w)
    a_w = self.sigmoid(a_w).view(b, c, 1, w)
    # Reweight the original feature map.
    return x * a_h * a_w

```

5.7 DEFINING INVERTED RESIDUAL BLOCK WITH ELA

```

#####
# 2. Define Inverted Residual Block with ELA-S (InvertedResidualELA_S)
#####
class InvertedResidualELA_S(nn.Module):
    """
    Inverted Residual block for MobileNetV3 that uses ELA-S in place of SE when use_se is True.
    """
    def __init__(
        self,
        cnf: InvertedResidualConfig,
        norm_layer: Callable[..., nn.Module],
        attn_layer: Optional[Callable[..., nn.Module]] = None # We'll pass ELA_S here.
    ):
        super().__init__()
        if not (1 <= cnf.stride <= 2):
            raise ValueError("illegal stride value")
        # Use residual connection if stride == 1 and input channels match output channels.
        self.use_res_connect = cnf.stride == 1 and cnf.input_channels == cnf.out_channels
        layers: List[nn.Module] = []
        activation_layer = nn.Hardswish if cnf.use_hs else nn.ReLU

        # Expansion phase.
        if cnf.expanded_channels != cnf.input_channels:
            layers.append(nn.Sequential(
                nn.Conv2d(cnf.input_channels, cnf.expanded_channels, kernel_size=1, bias=False),
                norm_layer(cnf.expanded_channels),
                activation_layer(inplace=True)
            ))

```

```

# Depthwise convolution.
stride = 1 if cnf.dilation > 1 else cnf.stride
layers.append(nn.Sequential(
    nn.Conv2d(cnf.expanded_channels, cnf.expanded_channels,
              kernel_size=cnf.kernel,
              stride=stride,
              padding=cnf.kernel // 2,
              dilation=cnf.dilation,
              groups=cnf.expanded_channels,
              bias=False),
    norm_layer(cnf.expanded_channels),
    activation_layer(inplace=True)
))
# Attention module: if use_se is True, insert our ELA-S.
if cnf.use_se:
    if attn_layer is not None:
        layers.append(attn_layer(cnf.expanded_channels))
    else:
        from torchvision.models.mobilenetv3 import SqueezeExcitation as SELayer
        from torchvision.models.mobilenetv3 import _make_divisible
        squeeze_channels = _make_divisible(cnf.expanded_channels // 4, 8)
        layers.append(SELayer(cnf.expanded_channels, squeeze_channels))
# Projection phase.
layers.append(nn.Sequential(
    nn.Conv2d(cnf.expanded_channels, cnf.out_channels, kernel_size=1, bias=False),
    norm_layer(cnf.out_channels)
))
self.block = nn.Sequential(*layers)
self.out_channels = cnf.out_channels

def forward(self, x: Tensor) -> Tensor:
    out = self.block(x)
    if self.use_res_connect:
        out += x
    return out

```

5.8 BUILDING MOBILENETV3 WITH ELA-S MODULE [PROPOSED MODEL]

```

#####
# 3. Build MobileNetV3 Large with ELA-S Module
#####
def mobilenet_v3_ela_s_large(*, weights: Optional[Any] = None, progress: bool = True, **kwargs: Any) -> MobileNetV3:
    """
    Constructs a MobileNetV3 large network that integrates the ELA-S module in place of the original SE blocks.

    Args:
        weights: Optional pretrained weights.
        progress: If True, display a progress bar.
        **kwargs: Additional arguments for MobileNetV3.
    """
    # Retrieve configuration for MobileNetV3 large.
    inverted_residual_setting, last_channel = _mobilenet_v3_conf("mobilenet_v3_large", **kwargs)
    # Use our custom block with ELA-S; replace blocks with use_se==True.
    block = lambda cnf, norm_layer: InvertedResidualELA_S(cnf, norm_layer, attn_layer=ELA_S)
    model = MobileNetV3(inverted_residual_setting, last_channel, block=block, **kwargs)
    # Optionally load pretrained weights (using strict=False to ignore key mismatches).
    if weights is not None:
        state_dict = weights.get_state_dict(progress=progress, check_hash=True)
        model.load_state_dict(state_dict, strict=False)
    return model

```

5.9 HELPER FUNCTION FOR FREEZING AND UNFREEZING LAYERS

```
# ===== Helper Functions for Freezing / Unfreezing =====
def freeze_all_except_classifier_and_attention(model):
    """
    Freeze all parameters in the model except for:
    - Those in the classifier.
    - Those in any attention module (i.e. instances of CoordAtt).
    """
    # Freeze everything first.
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze classifier parameters.
    for name, param in model.named_parameters():
        if "classifier" in name:
            param.requires_grad = True

    # Recursively unfreeze attention modules.
    def unfreeze_attention(module):
        for child in module.children():
            # Change this if your attention module has a different class name.
            if isinstance(child, ELA_S):
                for p in child.parameters():
                    p.requires_grad = True
            else:
                unfreeze_attention(child)
    unfreeze_attention(model)
```

5.10 TRAINING PHASE 1 [WITHOUT FREEZING LAYERS]

Function used for unfreezing the layers

```
def unfreeze_last_n_layers_and_attention(model, n):
    """
    Unfreeze the last 'n' parameters (in the order returned by model.named_parameters())
    and ensure that attention (CoordAtt) modules remain trainable.
    """
    # Unfreeze the last n parameters.
    params = list(model.named_parameters())
    for name, param in params[-n:]:
        param.requires_grad = True

    # Also ensure that any parameter from a CoordAtt module is trainable.
    def unfreeze_attention(module):
        for child in module.children():
            if isinstance(child, ELA_S):
                for p in child.parameters():
                    p.requires_grad = True
            else:
                unfreeze_attention(child)
    unfreeze_attention(model)

def unfreeze_all_layers(model):
    for param in model.parameters():
        param.requires_grad = True
```

Function to Print Model Summary

```
# ===== Function to Print Model Summary =====
def print_model_summary(model, input_size=(1, 3, 256, 256)):
    print("\n==== Model Summary ====")
    print(summary(model, input_size=input_size))
    print("\n=====")
```

```
=====
Total params: 2,872,914
Trainable params: 1,413,242
Non-trainable params: 1,459,672
Total mult-adds (Units.MEGABYTES): 282.86
=====
Input size (MB): 0.79
Forward/backward pass size (MB): 93.53
Params size (MB): 11.49
Estimated Total Size (MB): 105.81
=====
```

5.11 FUNCTION TO SHOW TRUE VS PREDICTED IMAGES AND GRAD-CAM VISUALIZATIONS

```
# ===== Function to Show True vs Predicted Images and GradCAM Visualizations =====
def show_visualizations(model, test_loader, device, class_names):
    print("\nTrue vs Predicted Images:")
    plot_true_vs_predicted(model, test_loader, device, class_names, model_path="best_model.pth")
    print("\nGrad-CAM Visualizations:")
    show_gradcam_examples(model, test_loader, device, class_names, model_path="best_model.pth")
```

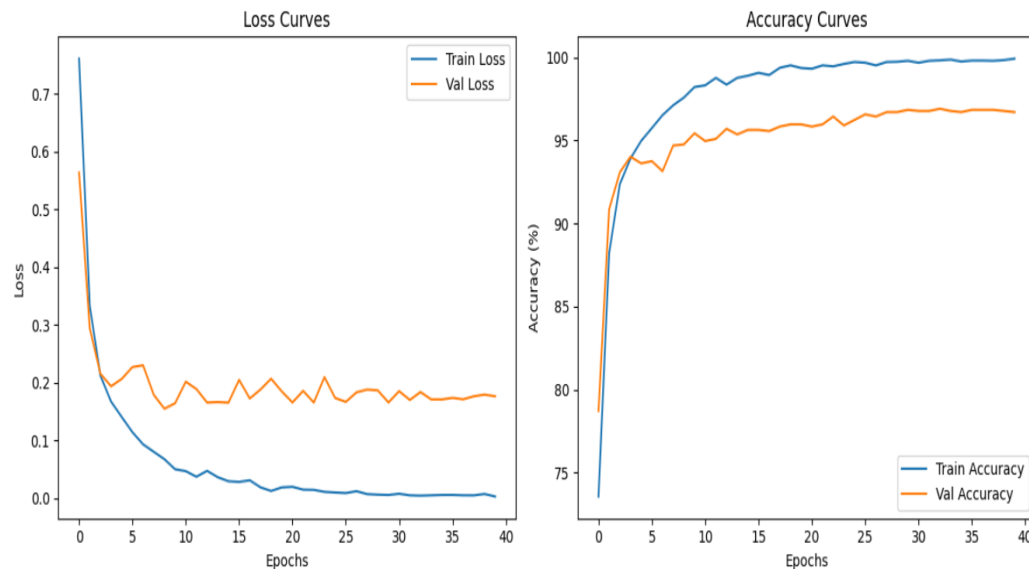
5.12 PHASE 1: TRAINING ONLY THE CLASSIFIER

```
# ===== Stage 1: Train Only the Classifier =====
print("Stage 1: Training only the classifier layer.")

# Freeze everything except the classifier
freeze_all_except_classifier_and_attention(model)
print_model_summary(model)
```

[Show hidden output](#)

Epoch 36/40	Train Loss: 0.0059 Acc: 99.81%	Val Loss: 0.1740 Acc: 96.84%	Time: 33.71s	204.88 samples/s
Epoch 37/40	Train Loss: 0.0053 Acc: 99.81%	Val Loss: 0.1715 Acc: 96.84%	Time: 34.72s	198.88 samples/s
Epoch 38/40	Train Loss: 0.0052 Acc: 99.80%	Val Loss: 0.1767 Acc: 96.84%	Time: 34.04s	202.88 samples/s
Epoch 39/40	Train Loss: 0.0076 Acc: 99.84%	Val Loss: 0.1796 Acc: 96.78%	Time: 34.48s	200.30 samples/s
Epoch 40/40	Train Loss: 0.0033 Acc: 99.93%	Val Loss: 0.1767 Acc: 96.71%	Time: 35.31s	195.55 samples/s



5.13 EVALUATION AFTER PHASE 1

```
# Evaluate after Stage 1
evaluate_model(model, test_loader, device, class_names)
#show_visualizations(model, test_loader, device, class_names)|
```

Test Accuracy: 97.28%

Classification Report:

	precision	recall	f1-score	support
betel_Bacterial Leaf Disease	0.96	0.95	0.95	148
betel_Dried Leaf	1.00	1.00	1.00	148
betel_Fungal Brown Spot Disease	0.98	1.00	0.99	148
betel_Healthy Leaf	0.96	0.95	0.96	148
tea_algal_spot	0.97	1.00	0.99	150
tea_brown_blight	0.89	0.98	0.93	130
tea_gray_blight	0.99	0.90	0.94	150
tea_healthy	0.99	0.99	0.99	150
tea_helopeltis	1.00	0.99	0.99	150
tea_red_spot	0.98	0.98	0.98	150
accuracy			0.97	1472
macro avg	0.97	0.97	0.97	1472
weighted avg	0.97	0.97	0.97	1472

5.14 FINE TUNING PHASE 2 [FREEZING LAST 20 LAYERS]

```
# ===== Stage 2: Fine-Tune the Last 20 Layers =====
n_layers_to_unfreeze = 20 # Adjust as needed
print(f"\nStage 2: Fine-tuning the last {n_layers_to_unfreeze} layers.")

unfreeze_last_n_layers_and_attention(model, n_layers_to_unfreeze)
print_model_summary(model)
```

Stage 2: Fine-tuning the last 20 layers.

==== Model Summary =====

Layer (type:depth-idx)	Output Shape	Param #
MobileNetV3	[1, 10]	--
├─Sequential: 1-1	[1, 960, 8, 8]	--
│ └─Conv2dNormActivation: 2-1	[1, 16, 128, 128]	--
│ └─Conv2d: 3-1	[1, 16, 128, 128]	(432)
│ └─BatchNorm2d: 3-2	[1, 16, 128, 128]	(32)

5.15 PHASE 2 SUMMARY

Total params: 2,872,914
Trainable params: 1,904,282
Non-trainable params: 968,632
Total mult-adds (Units.MEGABYTES): 282.86

=====
Input size (MB): 0.79
Forward/backward pass size (MB): 93.53
Params size (MB): 11.49
Estimated Total Size (MB): 105.81
=====

5.16 OPTIMIZER WITH THE UPDATED TRAINABLE PARAMETERS

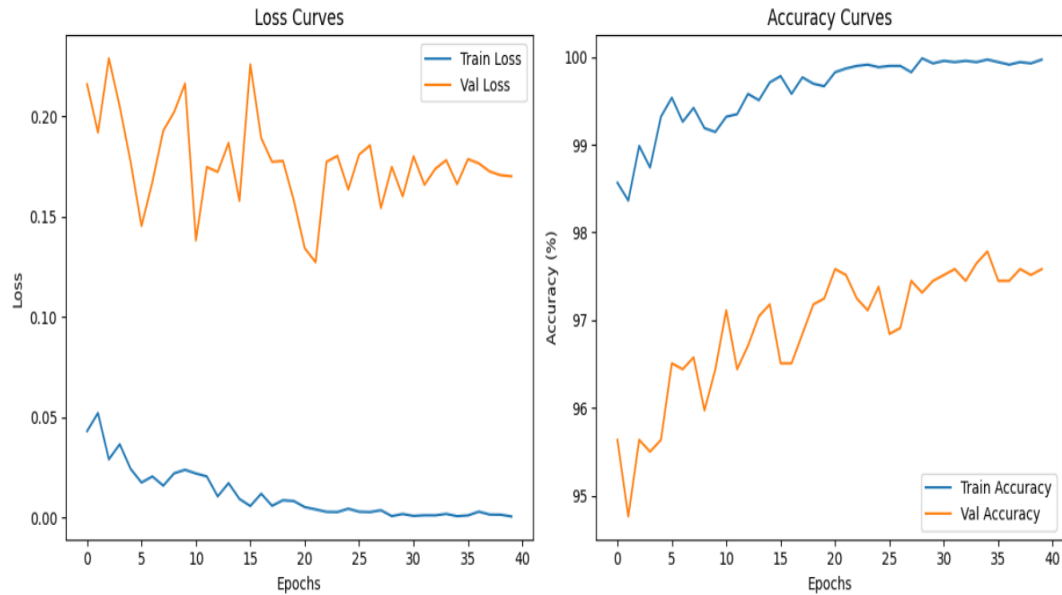
```
# Re-create the optimizer with the updated trainable parameters
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=learning_rate_initial)
scheduler = CosineAnnealingLR(optimizer, T_max=num_epochs_initial)
```

5.17 TRAINING PHASE 2

```
# Train phase 2
train_model(model, train_loader, val_loader, num_epochs=num_epochs_initial,
            optimizer=optimizer, criterion=criterion, device=device, scheduler=scheduler)
```

```
Epoch 1/40 | Train Loss: 0.0431 Acc: 98.57% | Val Loss: 0.2158 Acc: 95.63% | Time: 35.98s | 191.92 samples/s
Best model saved to best_model.pth
/usr/local/lib/python3.11/dist-packages/torch/optim/lr_scheduler.py:243: UserWarning: The epoch parameter in `scheduler.step()`
warnings.warn(EPOCH_DEPRECATION_WARNING, UserWarning)
Epoch 2/40 | Train Loss: 0.0521 Acc: 98.36% | Val Loss: 0.1918 Acc: 94.76% | Time: 34.77s | 198.61 samples/s
Epoch 3/40 | Train Loss: 0.0290 Acc: 98.99% | Val Loss: 0.2288 Acc: 95.63% | Time: 35.14s | 196.51 samples/s
Epoch 4/40 | Train Loss: 0.0365 Acc: 98.74% | Val Loss: 0.2047 Acc: 95.50% | Time: 35.81s | 192.85 samples/s
Epoch 5/40 | Train Loss: 0.0243 Acc: 99.32% | Val Loss: 0.1772 Acc: 95.63% | Time: 34.49s | 200.25 samples/s
Epoch 6/40 | Train Loss: 0.0175 Acc: 99.54% | Val Loss: 0.1453 Acc: 96.51% | Time: 35.10s | 196.75 samples/s
Best model saved to best_model.pth
Epoch 7/40 | Train Loss: 0.0205 Acc: 99.26% | Val Loss: 0.1672 Acc: 96.44% | Time: 34.51s | 200.12 samples/s
Epoch 8/40 | Train Loss: 0.0159 Acc: 99.42% | Val Loss: 0.1927 Acc: 96.57% | Time: 34.95s | 197.62 samples/s
Best model saved to best_model.pth
Epoch 9/40 | Train Loss: 0.0221 Acc: 99.19% | Val Loss: 0.2020 Acc: 95.97% | Time: 35.39s | 195.12 samples/s
Epoch 10/40 | Train Loss: 0.0238 Acc: 99.15% | Val Loss: 0.2162 Acc: 96.44% | Time: 34.05s | 202.81 samples/s
```

Epoch 37/40	Train Loss: 0.0030 Acc: 99.91%	Val Loss: 0.1764 Acc: 97.45%	Time: 34.59s	199.68 samples/s
Epoch 38/40	Train Loss: 0.0015 Acc: 99.94%	Val Loss: 0.1724 Acc: 97.58%	Time: 35.61s	193.93 samples/s
Epoch 39/40	Train Loss: 0.0015 Acc: 99.93%	Val Loss: 0.1705 Acc: 97.52%	Time: 34.28s	201.48 samples/s
Epoch 40/40	Train Loss: 0.0006 Acc: 99.97%	Val Loss: 0.1700 Acc: 97.58%	Time: 35.00s	197.33 samples/s



5.18 EVALUATION PHASE 2

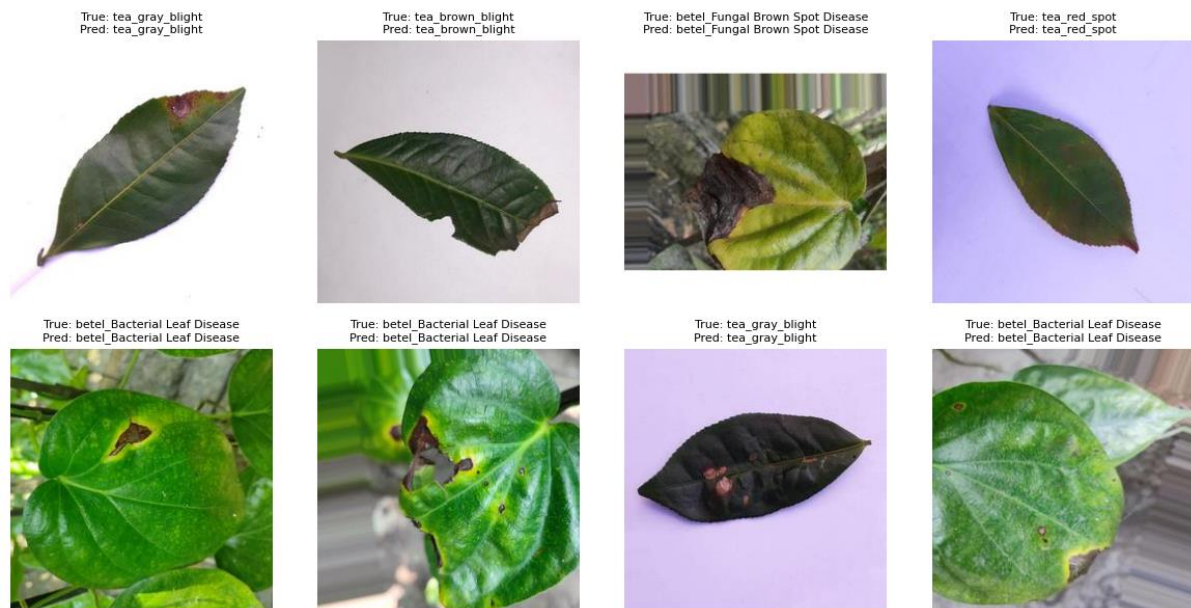
```
# Evaluate after Stage 2
evaluate_model(model, test_loader, device, class_names)
```

Test Accuracy: 98.51%

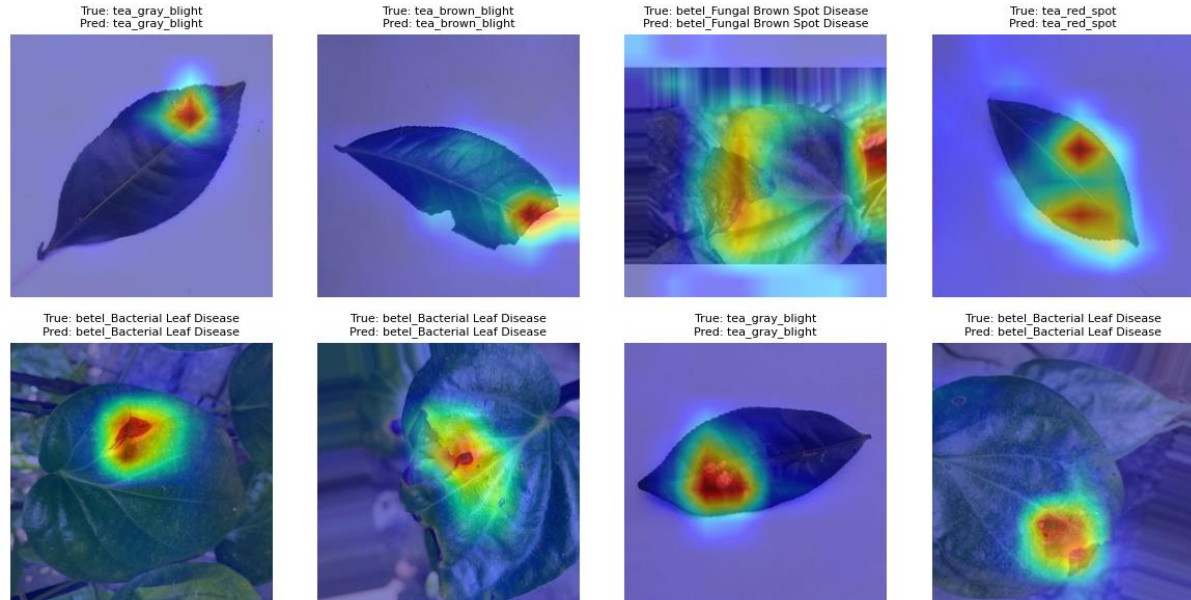
Classification Report:

	precision	recall	f1-score	support
betel_Bacterial Leaf Disease	0.97	0.95	0.96	148
betel_Dried Leaf	1.00	1.00	1.00	148
betel_Fungal Brown Spot Disease	0.99	1.00	0.99	148
betel_Healthy Leaf	0.97	0.97	0.97	148
tea_algal_spot	1.00	0.99	1.00	150
tea_brown_blight	0.96	0.98	0.97	130
tea_gray_blight	0.98	0.97	0.97	150
tea_healthy	0.99	0.99	0.99	150
tea_helopeltis	1.00	0.99	1.00	150
tea_red_spot	0.99	1.00	1.00	150
accuracy			0.99	1472
macro avg	0.98	0.99	0.98	1472
weighted avg	0.99	0.99	0.99	1472

5.19 PREDICTIONS



GRAD-CAM visualization images:



Chapter 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This study presents a lightweight and efficient deep learning approach for accurate classification of tea and betel leaf diseases, addressing a critical need for low-resource agricultural communities. The core contribution of this work is the development of a customized architecture—MobileNetV3 Small integrated with Efficient Local Attention (ELA)—which replaces the traditional Squeeze-and-Excitation (SE) block with a more computationally efficient ELA module to enhance feature aggregation.

To rigorously evaluate the effectiveness of the proposed model, extensive experiments were conducted using a well-balanced dataset of 10 leaf disease classes (six tea and four betel), each containing approximately 1,000 high-quality images. The evaluation involved both baseline and fine-tuned training phases for multiple architectures including:

- MobileNetV3 (Baseline and Fine-tuned) – 4,214,842 parameters
- MobileNetV3 + Coordinate Attention (CA) – 2,997,657 parameters
- MnasNet (Baseline and Fine-tuned) – 3,115,122 parameters
- Custom CNN – 228,632 parameters

Our proposed model, MobileNetV3 Small + ELA, not only achieved the highest test accuracy of 98.51% when fine-tuned but also maintained the lowest parameter count at 2,872,914, outperforming all other models in both performance and computational efficiency. This highlights the strength of the ELA module in enhancing attention-based feature extraction while maintaining a compact architecture.

By combining high accuracy with a reduced computational footprint, the MobileNetV3 + ELA model is particularly suited for deployment on mobile and edge devices, enabling real-time disease detection even in rural or resource-constrained environments. This accessibility makes it an invaluable tool for smallholder farmers, who often lack access to expert diagnostics or expensive agricultural technology.

In summary, this research demonstrates that integrating attention mechanisms like ELA into lightweight CNN architectures can significantly improve both classification accuracy and deployability. The proposed solution has strong potential to support early disease management, reduce crop losses, and contribute to more sustainable and resilient agricultural practices in developing regions.

6.2 LIMITATIONS AND FUTURE SCOPE

6.2.1 LIMITATIONS

Despite the strong performance and promising potential of the proposed MobileNetV3 + ELA model, this study has several limitations that should be acknowledged. Firstly, the dataset used, although balanced and high in quality, may not capture the full range of variability present in real-world farming conditions. Factors such as varying lighting, different stages of disease progression, diverse backgrounds, and weather-related distortions could affect model accuracy during practical deployment. Secondly, while the model has been optimized for mobile and edge devices in terms of computational efficiency, actual deployment and testing on physical hardware were not performed. As a result, critical real-time metrics such as latency, battery usage, and user interaction remain unverified.

Another limitation lies in the model's scope—being trained specifically for tea and betel leaf diseases, it may not generalize well to other crops or diseases without retraining on new data. Additionally, like many deep learning models, MobileNetV3 + ELA lacks interpretability, making it difficult for end-users, especially farmers without technical backgrounds, to understand the basis for its predictions. The model also depends heavily on high-quality images; blurry or poorly captured photos, which are common in rural or low-tech settings, could lead to misclassifications. Lastly, the current version of the system functions only as a classifier and does not offer a built-in feedback loop or adaptive learning mechanism to incorporate user corrections or field feedback over time.

6.2.2. FUTURE SCOPE

The success of the proposed MobileNetV3 + ELA model opens up several promising avenues for future development and large-scale impact. A key direction is the deployment of the trained model within cross-platform **mobile applications for Android and iOS**, allowing farmers to detect leaf diseases directly from their smartphones. This would provide an intuitive and accessible solution for on-the-go, real-time disease identification, even in remote locations with limited access to agricultural experts.

Another significant extension lies in the **integration of the model with Internet of Things (IoT) devices and agricultural drones**. By embedding the disease detection system into edge-enabled hardware, it can facilitate automated and real-time crop health monitoring across large plantations. Drones equipped with the model could periodically scan fields and identify affected areas with high precision, reducing manual labor and improving early intervention strategies.

Furthermore, the dataset can be continuously enriched by incorporating **newly emerging disease types** and capturing a wider range of image conditions, such as different seasons, growth stages, and environmental settings. This would help improve the model's robustness and ensure its adaptability to evolving agricultural challenges. Expanding the dataset and application scope will not only enhance the technical capabilities of the system but also increase its relevance and impact in the agricultural sector.

REFERENCES

1. Mohanty, S., Hughes, D., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, 1419. <https://doi.org/10.3389/fpls.2016.01419>
2. Sladojevic, S., Arsenovic, M., Anderla, A., Culibrk, D., & Stefanovic, D. (2016). Deep neural networks based recognition of plant diseases by leaf image classification. *Computational Intelligence and Neuroscience*, 2016, Article ID 3289801. <https://doi.org/10.1155/2016/3289801>
3. Too, J., Yujian, L., Njuki, S., & Yingchun, L. (2019). A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture*, 161, 272–279. <https://doi.org/10.1016/j.compag.2018.03.032>
4. Howard, A., et al. (2019). Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 1314–1324). <https://doi.org/10.1109/ICCV.2019.00140>
5. Hou, Q., Zhou, D., & Feng, J. (2021). Coordinate attention for efficient mobile network design. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 13708–13717). <https://doi.org/10.1109/CVPR46437.2021.01350>
6. Wang, X., Zhu, H., & Zhao, J. (2020). Efficient Layer Attention for deep convolutional neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(7), 10699–10706. <https://doi.org/10.1609/aaai.v34i07.6904>
7. Amara, J., Bouaziz, A., & Algergawy, H. (2017). A deep learning-based approach for banana leaf diseases classification. In *BTW Workshop on Big Data in Agriculture (CEUR Workshop Proceedings)*.
8. Zhang, K., Gong, B., Zhang, L., & Zhang, S. (2022). Tomato leaf disease recognition using a convolutional neural network model enhanced with attention mechanism. *Computers and Electronics in Agriculture*, 194, 106780. <https://doi.org/10.1016/j.compag.2022.106780>

9. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *arXiv preprint arXiv:1905.11946*. <https://doi.org/10.48550/arXiv.1905.11946>
10. Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A., & Le, Q. V. (2018). MnasNet: Platform-aware neural architecture search for mobile. *arXiv preprint arXiv:1807.11626*. <https://doi.org/10.48550/arXiv.1807.11626>
11. Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 7132–7141). <https://doi.org/10.1109/CVPR.2018.00745>
12. Hou, Q., Zhou, D., & Feng, J. (2021). Coordinate attention for efficient mobile network design. *arXiv preprint arXiv:2103.02907*. <https://doi.org/10.48550/arXiv.2103.02907>
13. Mei, K., Jiang, Z., Wu, Q., Li, S., & Li, M. (2018). An effective single-image super-resolution model using squeeze-and-excitation networks. *arXiv preprint arXiv:1810.01831*. <https://doi.org/10.48550/arXiv.1810.01831>
14. Chen, Y., Fan, X., & Zhang, L. (2021). Mobile-former: Bridging MobileNet and transformer. *arXiv preprint arXiv:2108.05895*. <https://doi.org/10.48550/arXiv.2108.05895>
15. Mehta, S., & Rastegari, M. (2021). MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer. *arXiv preprint arXiv:2110.02178*. <https://doi.org/10.48550/arXiv.2110.02178>
16. Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., & He, Q. (2021). A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1), 43–76. <https://doi.org/10.1109/JPROC.2020.3004555>
17. Khan, S., Naseer, M., Hayat, M., Zamir, S. W., Khan, F. S., & Shah, M. (2021). Transformers in vision: A survey. *ACM Computing Surveys*, 54(10s), 1–41. <https://doi.org/10.1145/3469024>

18. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). <https://doi.org/10.1109/CVPR.2016.90>
19. Chen, J., Liu, Q., & Gao, L. (2019). Visual tea leaf disease recognition using a convolutional neural network model. *Symmetry*, *11*(3), 343. <https://doi.org/10.3390/sym11030343>
20. Dey, A. K., Sharma, M., & Meshram, M. R. (2022). Development of ANN and ANFIS classifier for betel leaf pathogen detection. *Journal of the Institution of Engineers (India): Series B*, *103*, 1555–1562. <https://doi.org/10.1007/s40031-022-00720-3>
21. Zhao, Y., Xie, Y., & Wang, L. (2023). TLDDM: An enhanced tea leaf pest and disease detection model based on YOLOv8. *Agronomy*, *15*(3), 727. <https://doi.org/10.3390/agronomy15030727>
22. Chen, X., Shi, R., Cao, Y., & Zhang, Z. (2022). Detection and identification of tea leaf diseases based on AX-RetinaNet. *Scientific Reports*, *12*, 6181. <https://doi.org/10.1038/s41598-022-10356-2>
23. Das, P., Kundu, S., & Mitra, P. (2023). Early betel leaf disease detection using vision transformer and deep learning algorithms. *Computers and Electronics in Agriculture*, *203*, 107399. <https://doi.org/10.1016/j.compag.2023.107399>
24. Islam, M. M., Rahman, M. A., & Hossain, F. (2024). Tea leaf disease detection using segment anything model and deep learning. *Computers and Electronics in Agriculture*, *212*, 107009. <https://doi.org/10.1016/j.compag.2024.107009>
25. Rahman, M. A., Ahmed, F., Islam, M. M., & Miah, R. Z. (2023). Tea leaf disease detection and identification based on YOLOv7 and deep learning. *Scientific Reports*, *13*(1), 33270. <https://doi.org/10.1038/s41598-023-40397-9>
26. Ahmed, F., Rahman, M. A., Islam, M. M., & Hossain, M. R. (2025). A comprehensive image dataset for accurate diagnosis of betel leaf diseases. *Data in Brief*, *45*, 108123. <https://doi.org/10.1016/j.dib.2024.108123>

27. Li, Y., Liu, Z., Wang, W., & Jiang, H. (2023). Machine learning-based tea leaf disease detection: A systematic review. *arXiv preprint arXiv:2311.03240*. <https://doi.org/10.48550/arXiv.2311.03240>
28. Saha, S., Das, A., & Roy, S. (2022). Early recognition of betel leaf disease using deep learning with depth-wise separable convolutions. *Journal of Plant Pathology*, 104(3), 789–798. <https://doi.org/10.1007/s42161-022-01011-7>